

claude-windows / 068560d3-1ca2-4387-981a-d2d6bca55414

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\068560d3-1ca2-4387-981a-d2d6bca55414\subagents\workflows\wf_ef85c152-821\agent-a677e6345f2fa8b53.jsonl`  
- SHA-256: `738cd94a673cc67ed576e645d02727a1698d769d3d145fec02f720371a3b0681`  
- Source modified: `2026-06-16T06:30:29+00:00`  
- Imported at: `2026-07-05T16:48:29+00:00`  
- project: `wf_ef85c152-821`  
- session_id: `068560d3-1ca2-4387-981a-d2d6bca55414`
```

Transcript

1. user (2026-06-16T06:26:59.416Z)

LENS = ROBUSTNESS to the concurrent quality track + future change. The fixtures pin a windowing preset (vt/mg/mwd/max_win) as a LITERAL, not the named SERVED constant. If a future PR changes the build_windows / trajectory_to_action_windows ALGORITHM (the windowing session shipped #187 bounded-windowing, #188/#190 net-crossings gate), does the characterization fail LOUDLY (good) or could it pass silently / error? Does the gate/windowing change interact with the CONTROL path at all? If distill_local.build_candidates or rally_gate signatures change, what breaks and is it caught? Is the pinned-literal-vs-SERVED divergence documented + safe? What happens when real fixtures (P3) + corpus growth arrive (runtime, the deferred M4 quality-floor)?

AUDIT the SHIPPED 'Golden Regression Fixtures' feature (4 merged PRs #186/#189/#191/#192) for any gap the per-PR verification missed.
Read files in the git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\gf-doc\ (master HEAD 8d3e185): backend/eval/golden_fixtures.py (the core); tests/test_golden_fixtures.py; tests/test_golden_real_fixtures.py; eval_baselines/fixtures/ (3 synthetic fixtures + manifest.json + README.md); .github/workflows/ci.yml (the golden-crossos job); docs/GOLDEN_REGRESSION_FIXTURES.md (\$4c/\$5/\$6 claims). For analysis also read backend/eval/{experiment,distill_local,metrics,gt_loader,regression}.py and backend/pipeline/detectors/{tracknet_runner,rally_gate}.py.
THE FEATURE: a video-free regression harness. replay_fixture = parse_trajectory_csv -> distill_local.build_candidates(PINNED preset) -> VideoCandidates(5 shuttle cols only) -> experiment.predict(CONTROL, pure-stdlib) -> metrics.score. Synthetic Tier-0 fixtures + characterization (parse-compare: exact on int/structural, approx 1e-6 on float). freeze_real_fixture = de-attribution (random fx_<hex> slug, metadata strip, metric-invariant time-origin reset, STRICT gt_loader, positive no-person/audio/Md5/source-path scan) + the M2 refusal gate (raise unless owned AND minors_free AND non-blank license). The golden-crossos CI job runs `--check` + tests/test_golden_fixtures.py on ubuntu/windows/macOS.
ALREADY VERIFIED ? do NOT re-report as new findings: full suite 759 pass/7 skip; coverage 80.49%; ruff+mypy clean; cross-OS green on all 3 OSes for the COMMITTED fixtures; CONTROL avoids numpy *execution*.
Find what that MISSED: real gaps / bugs / edge-cases / privacy holes / determinism holes / over-claims. Cite file:line. Do NOT invent issues ? if a lens is clean, say so honestly and set verdict accordingly.

2. assistant (2026-06-16T06:27:02.570Z)

I'll audit the shipped Golden Regression Fixtures feature. Let me start by reading the core files in the worktree.

3. user (2026-06-16T06:27:04.286Z)

8d3e185 feat(eval): golden fixtures P1 ? de-attribution/minimization tool + M2 refusal gate (#192)
On branch feat/golden-fixtures-doc

Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean

4. user (2026-06-16T06:27:04.908Z)

```
1      """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3      `docs/GOLDEN_REGRESSION_FIXTURES.md` (§4, §6, §7). This module is the **single shared
4      replay code path** used by both the freeze tool and the characterization gate, and it
5      deliberately stays on the **pure-stdlib CONTROL path** (D5 in that doc): the learned
6      variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7      across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8      only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9      merges overlaps, and uses **no numpy**.
10
11     The replay reuses the already-shipped eval stack VERBATIM::
12
13         parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
14         ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
15         ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)
16
17     Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
18
19     * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
20     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
21     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true`,
``license="synthetic"`.
22     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
23     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
24     * The windowing preset params ``(vt, mg, mwd, max_win)`` are **pinned inside each
fixture's
25     provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the
named
26     ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn
fixtures.
27
28     §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
29     suite proves only that the deterministic post-trajectory transforms are unchanged for
the
30     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
31     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
32     field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
33
34     Pure + offline + stdlib. The replay imports `tracknet_runner` (which is itself stdlib-
only
35     for the parse/window math ? no TF), but pulls in no numpy/cv2 on the CONTROL path.
36     """
37     from __future__ import annotations
38
39     import argparse
40     import json
41     import math
42     import os
43     import re
44     import sys
45     from dataclasses import dataclass
46     from pathlib import Path
```

```

47     from typing import Any, Dict, List, Optional, Sequence, Tuple
48
49     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----
50     from backend.eval import distill_local
51     from backend.eval.experiment import CONTROL, VideoCandidates, predict
52     from backend.eval.gt_loader import load_gt_intervals
53     from backend.eval.metrics import score
54     from backend.eval.regression import gt_hash
55     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
56
57     Interval = Tuple[float, float]
58
59     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
60     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
61     CURRENT_SCHEMA = 1
62
63     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
64     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
65     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
66
67     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
68     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
69     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
70     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
71     #: re-states these in its provenance; the replay reads them from there.
72     PINNED_VT = 0.02
73     PINNED_MERGE_GAP = 2.0
74     PINNED_MIN_WINDOW = 2.0
75     PINNED_MAX_WINDOW = 0.0 # 0 = bounded-windowing OFF (W1), matching the SERVED default
76
77     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
78     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
79     SHUTTLE_FEATURES: Tuple[str, ...] = (
80         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
81     )
82     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
83         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
84     )
85
86     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
87     #: anywhere in a Tier-0 fixture (D3 / §4c). The privacy assertion scans for these.
88     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
89
90     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
91     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
92
93     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
94     #: emit a fixture whose written text contains one (it would be an attribution back-
channel, §4c/§5).
95     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
96
97
98     # =====
99     # Synthetic trajectory generation (deterministic ? NO random)

```

```

100 # =====
101
102
103 @dataclass(frozen=True)
104 class RallySpec:
105     """One in-flight rally segment: the shuttle oscillates across the net midline.
106
107     ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
108     period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
109     net midline along the play axis. Deterministic sine motion ? fixed
110     velocity/crossings.
111     """
112     start_frame: int
113     n_frames: int
114     period: float = 30.0
115     amplitude: float = 400.0
116
117
118 @dataclass(frozen=True)
119 class GapSpec:
120     """Dead-time between rallies ? what separates one window from the next.
121
122     ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
123     ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
124     end an active run so the next rally forms a distinct window.
125     """
126
127     start_frame: int
128     n_frames: int
129     kind: str = "still" # "still" | "occluded"
130
131
132 @dataclass(frozen=True)
133 class FixtureSpec:
134     """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
135
136     ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
137     "labels.csv". For a synthetic fixture they line up with the rallies by construction,
138     which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
139     field-quality one (§6).
140     """
141
142     slug: str
143     description: str
144     fps: float
145     frame_width: float
146     frame_height: float
147     segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)
148     gts: Tuple[Interval, ...]
149     net_midline: float = 640.0
150
151
152 def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
153                 ) -> List[Tuple[int, int, float, float]]:
154     """Deterministic in-flight shuttle: sine oscillation across the net midline (play
155     axis = x)."""
156     rows: List[Tuple[int, int, float, float]] = []
157     y_mid = frame_height / 2.0
158     for i in range(spec.n_frames):
159         x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
160         # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
161         unambiguously x.
162         y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
163         rows.append((spec.start_frame + i, 1, x, y))

```

```

162         return rows
163
164
165     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
166                    ) -> List[Tuple[int, int, float, float]]:
167         """Dead-time rows: resting-visible (still) or absent (occluded)."""
168         rows: List[Tuple[int, int, float, float]] = []
169         y_mid = frame_height / 2.0
170         for i in range(spec.n_frames):
171             if spec.kind == "occluded":
172                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
173             elif spec.kind == "still":
174                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
175             else: # pragma: no cover - guarded by builder
176                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
177         return rows
178
179
180     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
181         """Build the deterministic ``Frame, Visibility, X, Y`` rows for a synthetic
182         scenario."""
183         rows: List[Tuple[int, int, float, float]] = []
184         for seg in spec.segments:
185             if isinstance(seg, RallySpec):
186                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
187             elif isinstance(seg, GapSpec):
188                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
189             else: # pragma: no cover - guarded by dataclass shape
190                 raise TypeError(f"unknown segment type {type(seg).__name__}")
191         rows.sort(key=lambda r: r[0])
192         return rows
193
194     def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
195     None:
196         """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
197         floats."""
198         lines = ["Frame,Visibility,X,Y"]
199         for frame, vis, x, y in rows:
200             lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
201         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
202
203     def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
204         """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
205         LF."""
206         lines = ["start,end"]
207         for s, e in gts:
208             lines.append(f"{_fmt(s)},{_fmt(e)}")
209         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
210
211     # =====
212     # Built-in synthetic scenarios (the 3 committed slugs)
213     # =====
214
215     def _fr(seconds: float, fps: float) -> int:
216         return int(round(seconds * fps))
217
218
219     def builtin_specs() -> List[FixtureSpec]:
220         """The committed synthetic scenarios. Deterministic; chosen so each exercises a
221         distinct
222         windowing behaviour (one window / dead-time split / occlusion split)."""

```

```

222     fps = 30.0
223     fw, fh = 1280.0, 720.0
224     net = 640.0
225
226     # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
227     single = FixtureSpec(
228         slug="clean_single_rally",
229         description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
230         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
231         segments=(RallySpec(start_frame=0, n_frames=120),),
232         # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
233         gts=((0.0, 4.0),),
234     )
235
236     # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
237     # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
238     r0 = RallySpec(start_frame=0, n_frames=120)
239     gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
240     r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
241     multi = FixtureSpec(
242         slug="multi_rally_with_deadtime",
243         description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
244         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
245         segments=(r0, gap_still, r1),
246         gts=((0.0, 4.0), (7.0, 11.0)),
247     )
248
249     # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
250     # visibility break ? proves occlusion (not just low velocity) separates windows.
251     o0 = RallySpec(start_frame=0, n_frames=120)
252     gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
253     o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
254     occ = FixtureSpec(
255         slug="occlusion_break",
256         description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
257         fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
258         segments=(o0, gap_occ, o1),
259         gts=((0.0, 4.0), (7.0, 11.0)),
260     )
261
262     return [single, multi, occ]
263
264
265     def _spec_by_slug() -> Dict[str, FixtureSpec]:
266         return {s.slug: s for s in builtin_specs()}
267
268
269     # =====
270     # The shared replay (CONTROL / pure-stdlib path)
271     # =====
272
273
274     def _fmt(x: float) -> str:
275         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
``0.0``."""
276         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
277         if r == 0.0:
278             r = 0.0
279         return f"{r:.6f}"

```

```

280
281
282     def _round(x: float) -> float:
283         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
284         return 0.0 if r == 0.0 else r
285
286
287     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
288         """The windowing proposal tuple, read from the fixture provenance when present (the
pinned
289         params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""
290         if prov and "windowing" in prov:
291             w = prov["windowing"]
292             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
293         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
294
295
296     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
297         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
298
299         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
300         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
301
302         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
303         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
304
305         Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
306         control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
307         merges overlaps; NO numpy is touched.
308         """
309         fdir = _resolve_dir(slug_or_dir)
310         slug = fdir.name
311         traj_csv = fdir / "trajectory.csv"
312         labels_csv = fdir / "labels.csv"
313         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
314
315         fps, fw = _fps_fw(prov, slug)
316         iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
317         proposal = _pinned_proposal(prov)
318
319         intervals, feature_rows = distill_local.build_candidates(
320             str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
321
322         # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
323         # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
324         # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
325         features: Dict[str, List[float]] = {
326             name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
327         }
328         gts = load_gt_intervals(str(labels_csv), strict=True)
329         vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
330
331         preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
332         sc = score(preds, gts, iou)
333

```

```

334     candidates: List[Dict[str, Any]] = []
335     for j, (s, e) in enumerate(intervals):
336         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
337         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
338         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
339         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
340
341     return {
342         "slug": slug,
343         "schema_version": CURRENT_SCHEMA,
344         "iou": _round(iou),
345         "candidates": candidates,
346         "control_segments": [[_round(s), _round(e)] for s, e in preds],
347         "score": {
348             "iou_threshold": _round(sc.iou_threshold),
349             "num_pred": int(sc.num_pred),
350             "num_gt": int(sc.num_gt),
351             "true_positives": int(sc.true_positives),
352             "false_positives": int(sc.false_positives),
353             "false_negatives": int(sc.false_negatives),
354             "precision": _round(sc.precision),
355             "recall": _round(sc.recall),
356             "f1": _round(sc.f1),
357             "mean_iou": _round(sc.mean_iou),
358             "mean_start_error": _round(sc.mean_start_error),
359             "mean_end_error": _round(sc.mean_end_error),
360         },
361         "gt_hash": gt_hash(gts),
362     }
363
364
365     # =====
366     # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
367     # =====
368
369
370     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
371                     ) -> Dict[str, Any]:
372         """The committed provenance block ? pins the windowing + privacy/licensing tags
($4c, $5)."""
373         desc = ""
374         if isinstance(spec_or_dir, FixtureSpec):
375             desc = spec_or_dir.description
376         return {
377             "slug": slug,
378             "schema_version": CURRENT_SCHEMA,
379             "kind": "synthetic",
380             "license": "synthetic",
381             "minors_free": True,
382             "description": desc,
383             "fps": _round(fps),
384             "frame_width": _round(fw),
385             "iou": _round(iou),
386             "windowing": {
387                 "vt": PINNED_VT,
388                 "mg": PINNED_MERGE_GAP,
389                 "mwd": PINNED_MIN_WINDOW,
390                 "max_win": PINNED_MAX_WINDOW,
391             },
392             "paths": {
393                 "trajectory": "trajectory.csv",
394                 "labels": "labels.csv",

```

```

395         "expected": "expected.json",
396     },
397     "note": (
398         "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
399         "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
($6).",
400     ),
401 }
402
403
404 def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
405                   iou: float = 0.5) -> Dict[str, Any]:
406     """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
COMMITTED
407     ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
construction).
408
409     Idempotent: rewrites the same bytes given the same committed inputs. Requires the
trajectory
410     + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
411     """
412     fdir = fixtures_dir / slug
413     traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
414     if not traj_csv.is_file() or not labels_csv.is_file():
415         raise FileNotFoundError(
416             f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
generate them")
417
418     # Provenance first (it pins the windowing the replay reads), then replay ? expected.
419     spec = _spec_by_slug().get(slug)
420     fps, fw = (spec.fps, spec.frame_width) if spec is not None else
_fps_fw_from_prov(fdir, slug)
421     prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
422     _write_json(fdir / "provenance.json", prov)
423
424     expected = replay_fixture(fdir)
425     _write_json(fdir / "expected.json", expected)
426     return expected
427
428
429 def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
430     """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
expected.json +
431     provenance.json, and (re)write the manifest. Returns the slugs frozen."""
432     specs = builtin_specs()
433     fixtures_dir.mkdir(parents=True, exist_ok=True)
434     slugs: List[str] = []
435     for spec in specs:
436         fdir = fixtures_dir / spec.slug
437         fdir.mkdir(parents=True, exist_ok=True)
438         rows = make_synthetic_trajectory(spec)
439         write_trajectory_csv(rows, fdir / "trajectory.csv")
440         write_labels_csv(spec.gts, fdir / "labels.csv")
441         freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
442         slugs.append(spec.slug)
443     _write_manifest(specs, fixtures_dir)
444     return slugs
445
446
447 def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
448                     fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
449     """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
labels.csv ?
450     NO regeneration of the trajectory/labels. ? $6: this can rubber-stamp a real
regression; the

```

```

451 mitigation is reviewing the visible diff + recording a reason (the CLI requires
one)."""
452 if all_slugs:
453     targets = [d.name for d in sorted(fixture_dir.iterdir())
454                 if d.is_dir() and (d / "trajectory.csv").is_file()]
455 elif slug:
456     targets = [slug]
457 else:
458     raise ValueError("refresh_snapshot needs --slug S or --all")
459 for s in targets:
460     freeze_fixture(s, fixture_dir=fixture_dir)
461 return targets
462
463
464 # =====
465 # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
466 # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
467 # $4c anti-attribution, $5 threat model, $7 row P1, $6 honest limits.)
468 # =====
469
470
471 class RefusalError(ValueError):
472     """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
473
474     A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
475     it) that
476     names *why* the unsafe public-fixture path was refused. The refusal is the code-
477     enforced
478     provenance gate ($4c "provenance hard-fail gate", $7 ?): the owner-recorded /
479     minors-excluded
480     / license flags are human-asserted, but the gate makes the unsafe path impossible to
481     reach
482     *silently* ? a fixture cannot be written unless all three are affirmatively set.
483     """
484
485     def _new_surrogate_slug() -> str:
486         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` ($4c).
487
488         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
489         random,
490         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
491         over a
492         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
493         off-git.
494         """
495         return f"fx_{os.urandom(8).hex()}"
496
497     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). Source-
498     path tokens
499     #: that collide with these are NOT a leak ? exclude them so a short source stem like
500     ``traj`` can't
501     #: false-positive against the literal
502     ``trajectory.csv``/``labels.csv``/``expected.json``.
503     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
504         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
505         "provenance", "provenance.json",
506     )
507
508     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
509         """POSITIVE privacy assertion ($4c, red-team ?): raise if ``text`` leaks anything
510         attributable.
511         """

```

```

504     Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
505     (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
506     (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
507     always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
508     re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
509     """
510     low = text.lower()
511     for key in FORBIDDEN_KEYS:
512         if key in low:
513             raise RefusalError(
514                 f"{where}: forbidden key substring {key!r} found in written output ? "
515                 f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
516                 f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
517     m = _MD5_RE.search(text)
518     if m:
519         raise RefusalError(
520             f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
521             f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
522     for sp in source_paths:
523         if not sp:
524             continue
525         p = Path(sp)
526         for token in (sp, p.name, p.stem):
527             t = (token or "").strip().lower()
528             # Skip a token that is itself a substring of a schema-literal the writers
always emit
529             # (e.g. a source stem ``traj`` ? ``trajectory.csv``) ? that collision is not
a leak.
530             if not t or any(t in lit for lit in _SCHEMA_FILENAME_LITERALS):
531                 continue
532             if t in low:
533                 raise RefusalError(
534                     f"{where}: source-path token {token!r} found in written output ? the
source "
535                     f"filename/path must never be persisted (§4c metadata strip).")
536
537
538     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
539                               ) -> List[Tuple[int, int, float, float]]:
540         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
``(Frame,Visibility,X,Y)`` rows.
541
542         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
543         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
544         """
545         rows: List[Tuple[int, int, float, float]] = []
546         for p in points:
547             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
548         rows.sort(key=lambda r: r[0])
549         return rows
550
551
552     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
553         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant)."""

```

```

554         return [(s - t0_sec, e - t0_sec) for s, e in gts]
555
556
557     def freeze_real_fixture(
558         label: str,
559         trajectory_csv_path: Any,
560         labels_csv_path: Any,
561         *,
562         license: str,
563         minors_free: bool,
564         owned: bool,
565         fps: float,
566         frame_width: float,
567         fixtures_dir: Path = FIXTURES_DIR,
568         slug: Optional[str] = None,
569         time_origin_reset: bool = True,
570         source_note: str = "",
571     ) -> Dict[str, Any]:
572         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
573         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. (§4c / §5 / §7
row P1.)
574
575         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
576         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string.
577         The committed bytes carry NO filename / video_id / match / player / capture-date ?
only an
578         opaque random surrogate slug, the shuttle-only trajectory (time-origin reset by
default), the
579         reset labels, and a ``kind="cleared_real"`` provenance block. A positive privacy
assertion
580         re-reads the written ``expected.json`` + ``provenance.json`` and refuses if anything
attributable
581         slipped in.
582
583         NOTE (§6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
584         only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
585         attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
586         owner-asserted Fork-A / minors / biometric-exclusion flags.
587
588         Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
589         """
590         # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail (§4c / §7 ?)
-----
591         reasons: List[str] = []
592         if minors_free is not True:
593             reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
594                 "of a minor is prohibited ? §3, guardrail 'exclude minors')")
595         if owned is not True:
596             reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
597                 "broadcast/scraped = Fork B, do NOT commit ? §3 D1)")
598         if not (isinstance(license, str) and license.strip()):
599             reasons.append("license must be a non-blank string (per-record provenance tag ?
§4c "
600                 "'provenance hard-fail gate')")
601         if reasons:
602             raise RefusalError(

```

```

603         "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
604         "cure provenance/licensing; this gate sits behind counsel sign-off,
$6/P2):\n - "
605         + "\n - ".join(reasons))
606
607     traj_path = Path(str(trajjectory_csv_path))
608     labels_path = Path(str(labels_csv_path))
609
610     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
611     out_slug = slug if slug is not None else _new_surrogate_slug()
612
613     # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
-----
614     points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
615     if not points:
616         raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
617     gts = load_gt_intervals(str(labels_path), strict=True)
618
619     # --- (d) consistent time-origin reset (default ON; MUST be metric-invariant, $4c)
-----
620     # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
because
621     # metrics.score is built from pure differences (interval_iou, |start-start|, |end-
end|), the
622     # score is bit-identical with vs without reset ? only the absolute timeline is
severed. The
623     # quality-floor / characterization snapshot is therefore unchanged (asserted in the
tests).
624     if time_origin_reset:
625         t0_frame = min(int(p.frame) for p in points)
626         t0_sec = t0_frame / float(fps)
627     else:
628         t0_frame, t0_sec = 0, 0.0
629     rows = _reset_trajectory_rows(points, t0_frame)
630     reset_gts = _reset_labels(gts, t0_sec)
631
632     # --- (e) write the de-attributed fixture (REUSE M1 writers + replay_fixture)
-----
633     fdir = fixtures_dir / out_slug
634     fdir.mkdir(parents=True, exist_ok=True)
635     write_trajectory_csv(rows, fdir / "trajectory.csv")
636     write_labels_csv(reset_gts, fdir / "labels.csv")
637
638     iou = 0.5
639     prov = {
640         "slug": out_slug,
641         "schema_version": CURRENT_SCHEMA,
642         "kind": "cleared_real",
643         "label": str(label),
644         "license": license,
645         "minors_free": True,
646         "owned": True,
647         "fps": _round(fps),
648         "frame_width": _round(frame_width),
649         "iou": _round(iou),
650         "time_origin_reset": bool(time_origin_reset),
651         "windowing": {
652             "vt": PINNED_VT,
653             "mg": PINNED_MERGE_GAP,
654             "mwd": PINNED_MIN_WINDOW,
655             "max_win": PINNED_MAX_WINDOW,
656         },

```

```

657         "paths": {
658             "trajectory": "trajectory.csv",
659             "labels": "labels.csv",
660             "expected": "expected.json",
661         },
662         "gt_hash": gt_hash(reset_gts),
663         "source_note": str(source_note),
664         "note": (
665             "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
666             "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
667             "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
668             "Tier-0 / CONTROL-only. §6: behaviour-locking, not correctness; sits behind
counsel (P2).",
669         ),
670     }
671     _write_json(fdir / "provenance.json", prov)
672
673     expected = replay_fixture(fdir)          # REUSE M1 ? correct-by-construction
snapshot
674     _write_json(fdir / "expected.json", expected)
675
676     # --- (f) POSITIVE privacy assertion AFTER writing (scan the written text, §4c / ?)
-----
677     source_paths = [str(traj_path), str(labels_path)]
678     _scan_forbidden((fdir / "expected.json").read_text(encoding="utf-8"),
679                     where=f"{out_slug}/expected.json", source_paths=source_paths)
680     _scan_forbidden((fdir / "provenance.json").read_text(encoding="utf-8"),
681                     where=f"{out_slug}/provenance.json", source_paths=source_paths)
682
683     # --- (g) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
684     _append_manifest_entry(prov, fixtures_dir)
685     return expected
686
687
688     def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
689         """A manifest row for a single fixture, derived from its provenance block."""
690         slug = prov["slug"]
691         return {
692             "slug": slug,
693             "fps": _round(float(prov["fps"])),
694             "frame_width": _round(float(prov["frame_width"])),
695             "windowing": dict(prov.get("windowing", {})),
696             "kind": prov.get("kind", "synthetic"),
697             "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
698             "paths": {
699                 "dir": slug,
700                 "trajectory": f"{slug}/trajectory.csv",
701                 "labels": f"{slug}/labels.csv",
702                 "expected": f"{slug}/expected.json",
703                 "provenance": f"{slug}/provenance.json",
704             },
705         }
706
707
708     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
709         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
710         entries = load_manifest(fixtures_dir)
711         entries = [e for e in entries if e.get("slug") != prov["slug"]]
712         entries.append(_manifest_entry_for(prov))
713         entries.sort(key=lambda e: str(e.get("slug")))

```

```

714     _write_json(fixtures_dir / "manifest.json", entries)
715
716
717     # =====
718     # Manifest + loaders
719     # =====
720
721
722     def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
723         entries = [{
724             "slug": spec.slug,
725             "fps": _round(spec.fps),
726             "frame_width": _round(spec.frame_width),
727             "windowing": {
728                 "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
729                 "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
730             },
731             "kind": "synthetic",
732             "schema_version": CURRENT_SCHEMA,
733             "paths": {
734                 "dir": spec.slug,
735                 "trajectory": f"{spec.slug}/trajectory.csv",
736                 "labels": f"{spec.slug}/labels.csv",
737                 "expected": f"{spec.slug}/expected.json",
738                 "provenance": f"{spec.slug}/provenance.json",
739             },
740         } for spec in specs]
741         _write_json(fixtures_dir / "manifest.json", entries)
742
743
744     def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
745         """Load the fixtures manifest (returns [] if absent)."""
746         path = fixtures_dir / "manifest.json"
747         if not path.is_file():
748             return []
749         data = _read_json(path)
750         if not isinstance(data, list): # pragma: no cover - corrupt manifest
751             raise ValueError(f"{path}: manifest must be a JSON array")
752         return data
753
754
755     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
756         """Load one fixture's committed artifacts: `{provenance, expected, dir}`. Refuses
757         a fixture
758         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
759         fdir = fixtures_dir / slug
760         prov = _read_json(fdir / "provenance.json")
761         expected = _read_json(fdir / "expected.json")
762         sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
763         if sv > CURRENT_SCHEMA:
764             raise ValueError(
765                 f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade
766                 the harness")
767         return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
768
769     # =====
770     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
771     # =====
772
773     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
774     List[str]:
775         """Return a list of human-readable mismatch messages (empty == match).
776

```

```

776     Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
777     TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
778     equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
779     safe (the repo runs with core.autocrlf=true)."""
780     out: List[str] = []
781
782     def exact(path: str, a: Any, b: Any) -> None:
783         if a != b:
784             out.append(f"{path}: {a!r} != {b!r} (exact)")
785
786     def approx(path: str, a: Any, b: Any) -> None:
787         try:
788             if abs(float(a) - float(b)) > _FLOAT_TOL:
789                 out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
790         except (TypeError, ValueError):
791             out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
792
793     exact("slug", recomputed.get("slug"), committed.get("slug"))
794     exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
795     exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
796     approx("iou", recomputed.get("iou"), committed.get("iou"))
797
798     rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
799     if len(rc) != len(cc):
800         out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
801     else:
802         for i, (r, c) in enumerate(zip(rc, cc)):
803             approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
804             approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
805             rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
806             if set(rs) != set(cs):
807                 out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
808         else:
809             for k in rs:
810                 if k == "net_crossings":
811                     exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
812                 else:
813                     approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
814
815     rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
816     if len(rseg) != len(cseg):
817         out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
818     else:
819         for i, (r, c) in enumerate(zip(rseg, cseg)):
820             approx(f"control_segments[{i}][0]", r[0], c[0])
821             approx(f"control_segments[{i}][1]", r[1], c[1])
822
823     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
824     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
825     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
"mean_start_error", "mean_end_error")
826
827     for k in int_fields:
828         exact(f"score.{k}", rsc.get(k), csc.get(k))
829     for k in float_fields:
830         approx(f"score.{k}", rsc.get(k), csc.get(k))
831
832     return out
833

```

```

834
835 # =====
836 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
837 # =====
838
839
840 def _write_json(path: Path, obj: Any) -> None:
841     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
842     path.write_text(text + "\n", encoding="utf-8", newline="\n")
843
844
845 def _read_json(path: Path) -> Any:
846     with open(path, encoding="utf-8") as f:
847         return json.load(f)
848
849
850 def _resolve_dir(slug_or_dir: Any) -> Path:
851     if isinstance(slug_or_dir, Path):
852         return slug_or_dir
853     p = Path(str(slug_or_dir))
854     if p.is_dir():
855         return p
856     return FIXTURES_DIR / str(slug_or_dir)
857
858
859 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
860     if prov and "fps" in prov and "frame_width" in prov:
861         return float(prov["fps"]), float(prov["frame_width"])
862     spec = _spec_by_slug().get(slug)
863     if spec is not None:
864         return spec.fps, spec.frame_width
865     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
866
867
868 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:
869     prov_path = fdir / "provenance.json"
870     prov = _read_json(prov_path) if prov_path.is_file() else None
871     return _fps_fw(prov, slug)
872
873
874 # =====
875 # CLI
876 # =====
877
878
879 def _cmd_check(fixtures_dir: Path) -> int:
880     manifest = load_manifest(fixtures_dir)
881     if not manifest:
882         print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
file=sys.stderr)
883         return 1
884     failures = 0
885     for entry in manifest:
886         slug = entry["slug"]
887         committed = _read_json(fixtures_dir / slug / "expected.json")
888         recomputed = replay_fixture(fixtures_dir / slug)
889         mismatches = compare_expected(recomputed, committed)
890         if mismatches:
891             failures += 1
892             print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
f"(characterization = behaviour-locking, NOT correctness ? S6):")
893             for m in mismatches:
894                 print(f"    - {m}")
895         else:

```

```

897         sc = recomputed["score"]
898         print(f"[ok]   {slug}: {len(recomputed['candidates'])} candidates, "
899               f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
900     print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match
901         "
902         f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
903     return 0 if failures == 0 else 1
904
905     def main(argv: Optional[Sequence[str]] = None) -> int:
906         ap = argparse.ArgumentParser(
907             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
908             harness.")
909         g = ap.add_mutually_exclusive_group(required=True)
910         g.add_argument("--freeze-synthetic", action="store_true",
911             help="generate synthetic trajectory+labels for all built-in
912             scenarios, then "
913             "freeze expected.json+provenance.json + write manifest.json")
914         g.add_argument("--refresh-snapshot", action="store_true",
915             help="re-freeze expected.json from COMMITTED trajectory+labels (no
916             regeneration)")
917         g.add_argument("--check", action="store_true",
918             help="replay all fixtures + parse-compare vs committed expected.json
919             (exit 0/1)")
920         g.add_argument("--freeze-real", action="store_true",
921             help="(P1) de-attribute an OWNED real trajectory+labels into a
922             committable Tier-0 "
923             "fixture; REFUSED without --owned --minors-free and a non-blank
924             --license")
925         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
926             slug for "
927             "--freeze-real (default: a random fx_<hex> id)")
928         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
929             snapshot")
930         ap.add_argument("--reason", help="recorded reason for --refresh-snapshot (review the
931             diff!)")
932         # --freeze-real (P1 / M2 refusal gate) options:
933         ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
934             (Frame,Visibility,X,Y)")
935         ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
936             (start,end)")
937         ap.add_argument("--license", dest="license_", default=None,
938             help="--freeze-real) the source license tag (required, non-blank)")
939         ap.add_argument("--owned", action="store_true",
940             help="--freeze-real) assert owner-recorded Fork-A source
941             (required)")
942         ap.add_argument("--minors-free", dest="minors_free", action="store_true",
943             help="--freeze-real) assert the source contains NO minors
944             (required)")
945         ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
946         ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
947             real) source frame width (px)")
948         ap.add_argument("--label", default="cleared_real", help="--freeze-real) non-
949             identifying scenario label")
950         ap.add_argument("--source-note", dest="source_note", default="",
951             help="--freeze-real) non-identifying note (NEVER a path/name/id)")
952         ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
953             action="store_false",
954             help="--freeze-real) DISABLE the anti-attribution time-origin reset
955             (default ON)")
956         ap.set_defaults(time_origin_reset=True)
957         ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
958             help="override the fixtures directory (default: repo
959             eval_baselines/fixtures)")
960         args = ap.parse_args(list(argv) if argv is not None else None)

```

```

943
944     fixtures_dir: Path = args.fixtures_dir
945
946     if args.freeze_synthetic:
947         slugs = freeze_synthetic(fixtures_dir)
948         print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
949             '.join(slugs)}")
950         print(f"  -> {fixtures_dir}")
951         return 0
952
953     if args.refresh_snapshot:
954         if not args.slug and not args.all:
955             ap.error("--refresh-snapshot requires --slug S or --all")
956         if not args.reason:
957             print("[golden-fixtures] WARNING: no --reason given. --refresh-snapshot can
958                 "
959                 "rubber-stamp a real regression ? review the visible diff ($6).",
960                 file=sys.stderr)
961             targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
962                 fixtures_dir=fixtures_dir)
963             print(f"[golden-fixtures] refreshed snapshot for: {'', '.join(targets)}"
964                 + (f" (reason: {args.reason})" if args.reason else ""))
965             return 0
966
967     if args.freeze_real:
968         missing = [n for n, v in (("--trajectory", args.trajectory),
969             ("--labels", args.labels),
970             ("--fps", args.fps),
971             ("--frame-width", args.frame_width)) if v is None]
972         if missing:
973             ap.error("--freeze-real requires " + ", ".join(missing))
974         try:
975             expected = freeze_real_fixture(
976                 args.label, args.trajectory, args.labels,
977                 license=args.license_ if args.license_ is not None else "",
978                 minors_free=args.minors_free, owned=args.owned,
979                 fps=args.fps, frame_width=args.frame_width,
980                 fixtures_dir=fixtures_dir, slug=args.slug,
981                 time_origin_reset=args.time_origin_reset, source_note=args.source_note)
982         except RefusalError as e:
983             # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
984             zero.
985             print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
986                 file=sys.stderr)
987             return 2
988             print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'
989                 "
990                 f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
991             print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
992                 to the "
993                 "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
994                 (P2).")
995             return 0
996
997     # --check
998     return _cmd_check(fixtures_dir)
999
1000 if __name__ == "__main__":
1001     raise SystemExit(main())
1002

```

5. assistant (2026-06-16T06:27:09.008Z)

Now let me read the test files and the core dependencies the replay reuses.

6. user (2026-06-16T06:27:09.879Z)

```
1      """Golden Regression Fixtures (M1) ? characterization tests over the committed synthetic
Tier-0.
2
3      Parametrized over ``load_manifest()`` so every committed fixture is exercised. The gate
is
4      **parse-compare** (never byte-compare), **CONTROL-only** (pure stdlib, no numpy on the
replay
5      path), and asserts the Tier-0 privacy invariant positively (no
person/audio/pose/gait/face keys).
6
7      $6 honest-limits caveat: green here proves only that the deterministic post-trajectory
transforms
8      are unchanged for the frozen synthetic cases ? characterization is behaviour-locking,
NOT
9      correctness, and synthetic-tier numbers measure plumbing, not field quality.
10     """
11     import json
12
13     import pytest
14
15     from backend.eval import golden_fixtures as gf
16
17     # Slugs discovered from the committed manifest ? the suite grows by accretion
automatically.
18     MANIFEST = gf.load_manifest()
19     SLUGS = [entry["slug"] for entry in MANIFEST]
20
21
22     def test_manifest_is_non_empty():
23         """A committed manifest with at least one fixture must exist (else nothing is
gated)."""
24         assert SLUGS, "no fixtures in eval_baselines/fixtures/manifest.json ? run --freeze-
synthetic"
25
26
27     @pytest.mark.parametrize("slug", SLUGS)
28     def test_replay_matches_committed_expected(slug):
29         """Replay the fixture and parse-compare against its committed expected.json.
30
31         Characterization = behaviour-locking, NOT correctness ($6): a non-empty mismatch
list means
32         the deterministic post-trajectory behaviour CHANGED for this frozen case, not that
it is wrong.
33         """
34         committed = gf.load_fixture(slug)["expected"]
35         recomputed = gf.replay_fixture(slug)
36         mismatches = gf.compare_expected(recomputed, committed)
37         assert mismatches == [], (
38             f"{slug}: behaviour changed vs frozen snapshot (characterization, not
correctness ? $6):\n"
39             + "\n".join(f" - {m}" for m in mismatches)
40         )
41
42
43     @pytest.mark.parametrize("slug", SLUGS)
44     def test_schema_version_le_current(slug):
45         """Every committed fixture's schema_version must be <= CURRENT_SCHEMA (forward-
compatible)."""
46         fx = gf.load_fixture(slug)
47         assert int(fx["expected"]["schema_version"]) <= gf.CURRENT_SCHEMA
48         assert int(fx["provenance"]["schema_version"]) <= gf.CURRENT_SCHEMA
49
50
```

```

51     @pytest.mark.parametrize("slug", SLUGS)
52     def test_no_person_or_audio_keys(slug):
53         """Positive privacy assertion (D3 / §4c): NO biometric/identity/non-shuttle key
appears
54         anywhere in the committed expected.json ? not trusting a silent 0.0 default."""
55         fdir = gf.FIXTURES_DIR / slug
56         raw = (fdir / "expected.json").read_text(encoding="utf-8").lower()
57         for forbidden in gf.FORBIDDEN_KEYS:
58             assert forbidden not in raw, (
59                 f"{slug}: forbidden key substring {forbidden!r} present in expected.json
(Tier-0 leak)")
60
61         expected = gf.load_fixture(slug)["expected"]
62         for i, cand in enumerate(expected["candidates"]):
63             # The only feature block is the shuttle block, and it holds EXACTLY the 5
shuttle cols.
64             assert set(cand.keys()) == {"start", "end", "shuttle"}, f"{slug}: candidate[{i}]
has extra keys"
65             assert set(cand["shuttle"].keys()) == set(gf.SHUTTLE_FEATURES), (
66                 f"{slug}: candidate[{i}].shuttle is not exactly the 5 shuttle features")
67
68
69     @pytest.mark.parametrize("slug", SLUGS)
70     def test_synthetic_provenance_tags(slug):
71         """Tier-0 hard tags: kind=synthetic, license=synthetic, minors_free=true (constraint
5)."""
72         prov = gf.load_fixture(slug)["provenance"]
73         assert prov["kind"] == "synthetic"
74         assert prov["license"] == "synthetic"
75         assert prov["minors_free"] is True
76         # Windowing pinned in the fixture (not the named SERVED constant) ? constraint 3.
77         assert set(prov["windowing"]) == {"vt", "mg", "mwd", "max_win"}
78
79
80     def test_control_is_not_learned():
81         """The replay must use the pure-stdlib CONTROL variant (no numpy/LogisticRegression
? D5)."""
82         from backend.eval.experiment import CONTROL
83         assert CONTROL.is_learned() is False
84
85
86     def test_perturbing_trajectory_changes_segments(tmp_path):
87         """Mutating a copy of a trajectory and replaying yields DIFFERENT segments ? proving
the
88         windows are RE-DERIVED from the CSV each run, not merely echoed from
expected.json."""
89         slug = SLUGS[0]
90         src = gf.FIXTURES_DIR / slug
91         work = tmp_path / slug
92         work.mkdir()
93         # Copy the fixture, then TRUNCATE the trajectory to its first half so the derived
window's
94         # `end` shrinks ? a guaranteed structural change in the re-derived segments.
(Deleting an
95         # interior hole would not move start/end, since merge_gap coalesces across it and
the frame
96         # indices of the surviving rows are unchanged.)
97         lines = (src / "trajectory.csv").read_text(encoding="utf-8").splitlines()
98         header, body = lines[0], lines[1:]
99         mutated = [header] + body[: len(body) // 2]
100         (work / "trajectory.csv").write_text("\n".join(mutated) + "\n", encoding="utf-8",
newline="\n")
101         (work / "labels.csv").write_text((src / "labels.csv").read_text(encoding="utf-8"),
encoding="utf-8", newline="\n")
102         (work / "provenance.json").write_text((src /

```

```

"provenance.json").read_text(encoding="utf-8"),
104                                     encoding="utf-8", newline="\n")
105
106     original = gf.load_fixture(slug)["expected"]
107     perturbed = gf.replay_fixture(work)
108     assert perturbed["control_segments"] != original["control_segments"], (
109         "perturbed trajectory produced identical segments ? replay is echoing, not re-
deriving")
110
111
112     def test_freeze_is_idempotent(tmp_path):
113         """Re-freezing every built-in fixture into a temp dir produces expected.json
structurally
114         equal to the committed one (correct-by-construction + deterministic)."""
115         slugs = gf.freeze_synthetic(fixtures_dir=tmp_path)
116         assert slugs, "freeze_synthetic produced no fixtures"
117         for slug in slugs:
118             committed = gf.load_fixture(slug)["expected"]
119             refrozen = json.loads((tmp_path / slug /
"expected.json").read_text(encoding="utf-8"))
120             # Structural (parsed) equality ? no float churn, exact same shape/values.
121             assert gf.compare_expected(refrozen, committed) == [], (
122                 f"{slug}: re-frozen expected.json differs from committed (non-idempotent
freeze)")
123             assert refrozen == committed, f"{slug}: re-frozen expected.json not structurally
identical"
124
125
126     def test_check_cli_passes(capsys):
127         """The --check CLI replays all fixtures and exits 0 when they match the committed
snapshots."""
128         rc = gf.main(["--check"])
129         assert rc == 0
130

```

7. user (2026-06-16T06:27:10.310Z)

```

1     """Golden Regression Fixtures (P1 + M2) ? de-attribution / minimization + the refusal
gate.
2
3     These tests exercise ``golden_fixtures.freeze_real_fixture`` (the P1 tooling that turns
an OWNED,
4     minors-free real trajectory + golden labels into a committable, de-attributed Tier-0
fixture) and
5     its M2 ``--license`` / ``--minors-free`` / ``--owned`` refusal gate.
6
7     Everything runs on SYNTHETIC input written into ``tmp_path`` ? NO real data, NO network,
NO GPU.
8     We reuse M1's synthetic generators (``builtin_specs`` / ``make_synthetic_trajectory``
and the
9     ``write_trajectory_csv`` / ``write_labels_csv`` writers) to manufacture the "source"
CSVs, then
10    prove:
11        * the refusal gate blocks every unsafe combination (M2);
12        * a successful freeze is de-attributed (random ``fx_`` slug, no id/name/path, no
FORBIDDEN_KEYS);
13        * the time-origin reset is metric-INVARIANT (score identical with/without; only
absolute
14        ``control_segments`` shift toward 0);
15        * the freeze round-trips (``replay_fixture`` + ``compare_expected`` == []).
16
17    $6 honest-limits caveat: a green suite proves only that the deterministic post-
trajectory transforms
18    are unchanged for the frozen cases ? characterization is behaviour-locking, NOT
correctness.

```

```

19     """
20     import pytest
21
22     from backend.eval import golden_fixtures as gf
23
24
25     # -----
26     # Helpers ? manufacture a SYNTHETIC "real" source (trajectory + labels CSV) in tmp_path.
27     # -----
28
29
30     def _write_source(tmp_path, *, frame_offset: int = 0):
31         """Write a synthetic source trajectory+labels CSV pair into ``tmp_path``.
32
33         Reuses M1's first built-in scenario (a clean single rally), optionally shifted by
34         ``frame_offset`` frames (and the matching label seconds) to model an arbitrary
absolute
35         capture timeline. Returns ``(traj_path, labels_path, fps, frame_width, gts)``.
36         """
37         spec = gf.builtin_specs()[0] # clean_single_rally
38         fps, fw = spec.fps, spec.frame_width
39         rows = gf.make_synthetic_trajectory(spec)
40         if frame_offset:
41             rows = [(f + frame_offset, v, x, y) for (f, v, x, y) in rows]
42         gts = list(spec.gts)
43         if frame_offset:
44             off_sec = frame_offset / fps
45             gts = [(s + off_sec, e + off_sec) for (s, e) in gts]
46
47         traj = tmp_path / "src_trajectory.csv"
48         labels = tmp_path / "src_labels.csv"
49         gf.write_trajectory_csv(rows, traj)
50         gf.write_labels_csv(gts, labels)
51         return traj, labels, fps, fw, gts
52
53
54     # -----
55     # M2 ? refusal gate
56     # -----
57
58
59     @pytest.mark.parametrize(
60         "kwargs, why",
61         [
62             ({"minors_free": False, "owned": True, "license": "owned"},
"minors_free=False"),
63             ({"minors_free": True, "owned": False, "license": "owned"}, "owned=False"),
64             ({"minors_free": True, "owned": True, "license": ""}, "license=' '"),
65             ({"minors_free": True, "owned": True, "license": "   "}, "license=blank"),
66             ({"minors_free": True, "owned": True, "license": None}, "license=None"),
67         ],
68     )
69     def test_refusal_gate_blocks_unsafe(tmp_path, kwargs, why):
70         """freeze_real_fixture refuses (ValueError) unless minors_free AND owned AND a non-
blank license.
71
72         The refusal is the code-enforced provenance gate ($4c / $7 ?): the unsafe public-
fixture path
73         must be impossible to reach silently.
74         """
75         traj, labels, fps, fw, _ = _write_source(tmp_path)
76         with pytest.raises(ValueError) as ei:
77             gf.freeze_real_fixture(
78                 "scenario", traj, labels,
79                 fps=fps, frame_width=fw, fixtures_dir=tmp_path / "fixtures", **kwargs)

```

```

80         # RefusalError is a ValueError subclass; the message must name a reason.
81         assert isinstance(ei.value, gf.RefusalError), why
82         assert "REFUSED" in str(ei.value), why
83         # Nothing was written for a refused freeze.
84         assert not (tmp_path / "fixtures").exists() or not any((tmp_path /
"fixtures").iterdir()), (
85             f"{why}: a refused freeze must not write any fixture")
86
87
88         # -----
89         # P1 ? de-attribution
90         # -----
91
92
93     def test_real_fixture_is_deattributed(tmp_path):
94         """A successful safe freeze is de-attributed: kind=cleared_real, fx_-prefixed
surrogate slug,
95         no id/name/filename/source-path key, and NO FORBIDDEN_KEYS in expected.json."""
96         fixtures = tmp_path / "fixtures"
97         traj, labels, fps, fw, _ = _write_source(tmp_path)
98         expected = gf.freeze_real_fixture(
99             "clean_rally", traj, labels,
100             license="owned", minors_free=True, owned=True,
101             fps=fps, frame_width=fw, fixtures_dir=fixtures)
102
103         slug = expected["slug"]
104         assert slug.startswith("fx_"), f"surrogate slug must be opaque fx_<hex>, got
{slug!r}"
105
106         prov = gf._read_json(fixtures / slug / "provenance.json")
107         assert prov["kind"] == "cleared_real"
108         assert prov["minors_free"] is True and prov["owned"] is True
109         assert prov["license"] == "owned"
110         assert prov["slug"] == slug
111         # No attribution keys / no source filename anywhere in the provenance.
112         for banned in ("video_id", "name", "filename", "path", "match", "capture_date"):
113             assert banned not in prov, f"provenance must not carry an attribution key
{banned!r}"
114         prov_text = (fixtures / slug /
"provenance.json").read_text(encoding="utf-8").lower()
115         for tok in ("src_trajectory", "src_labels", str(traj).lower(), str(labels).lower()):
116             assert tok not in prov_text, f"source token {tok!r} leaked into provenance"
117
118         # expected.json carries no biometric/identity/non-shuttle stream.
119         exp_text = (fixtures / slug / "expected.json").read_text(encoding="utf-8").lower()
120         for forbidden in gf.FORBIDDEN_KEYS:
121             assert forbidden not in exp_text, f"FORBIDDEN_KEYS leak {forbidden!r} in
expected.json"
122
123         # The fixture is registered in the manifest with kind=cleared_real.
124         manifest = gf.load_manifest(fixtures)
125         rows = [e for e in manifest if e["slug"] == slug]
126         assert len(rows) == 1 and rows[0]["kind"] == "cleared_real"
127
128
129     def test_supplied_slug_is_honored(tmp_path):
130         """An explicitly supplied slug is used verbatim (the surrogate map is the caller's
concern)."""
131         fixtures = tmp_path / "fixtures"
132         traj, labels, fps, fw, _ = _write_source(tmp_path)
133         expected = gf.freeze_real_fixture(
134             "scenario", traj, labels, slug="fx_custom00",
135             license="owned", minors_free=True, owned=True,
136             fps=fps, frame_width=fw, fixtures_dir=fixtures)
137         assert expected["slug"] == "fx_custom00"

```

```

138         assert (fixtures / "fx_custom00" / "expected.json").is_file()
139
140
141     def test_freeze_real_does_not_clobber_existing_manifest(tmp_path):
142         """Freezing a second real fixture must APPEND to the manifest, not wipe the
143         first."""
144         fixtures = tmp_path / "fixtures"
145         traj, labels, fps, fw, _ = _write_source(tmp_path)
146         e1 = gf.freeze_real_fixture("a", traj, labels, slug="fx_aaaa0001",
147                                   license="owned", minors_free=True, owned=True,
148                                   fps=fps, frame_width=fw, fixtures_dir=fixtures)
149         e2 = gf.freeze_real_fixture("b", traj, labels, slug="fx_bbbb0002",
150                                   license="owned", minors_free=True, owned=True,
151                                   fps=fps, frame_width=fw, fixtures_dir=fixtures)
152         slugs = {e["slug"] for e in gf.load_manifest(fixtures)}
153         assert {e1["slug"], e2["slug"]} <= slugs
154
155     def test_positive_privacy_assertion_catches_leak_in_source_note(tmp_path):
156         """source_note is non-identifying by contract ? a path/name in it trips the positive
157         scan."""
158         fixtures = tmp_path / "fixtures"
159         traj, labels, fps, fw, _ = _write_source(tmp_path)
160         with pytest.raises(gf.RefusalError):
161             gf.freeze_real_fixture(
162                 "scenario", traj, labels, slug="fx_leak0001",
163                 license="owned", minors_free=True, owned=True,
164                 fps=fps, frame_width=fw, fixtures_dir=fixtures,
165                 source_note=str(traj)) # leaking the source path ? must be refused
166
167         # -----
168         # P1 (d) ? time-origin reset is metric-invariant
169         # -----
170
171
172     def test_time_origin_reset_is_metric_invariant(tmp_path):
173         """A large-offset source frozen WITH reset has IDENTICAL score to the SAME source
174         frozen
175         WITHOUT reset ? but its absolute control_segments shift toward 0 (severed
176         timeline)."""
177         offset_frames = 9000 # +5 minutes at 30fps ? a large absolute capture-timeline
178         offset
179         traj, labels, fps, fw, _ = _write_source(tmp_path, frame_offset=offset_frames)
180
181         with_reset = gf.freeze_real_fixture(
182             "scenario", traj, labels, slug="fx_reset_on",
183             license="owned", minors_free=True, owned=True,
184             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "with",
185             time_origin_reset=True)
186         without_reset = gf.freeze_real_fixture(
187             "scenario", traj, labels, slug="fx_reset_off",
188             license="owned", minors_free=True, owned=True,
189             fps=fps, frame_width=fw, fixtures_dir=tmp_path / "without",
190             time_origin_reset=False)
191
192         # (1) Score is bit-identical ? metrics.score is built from pure differences.
193         assert with_reset["score"] == without_reset["score"], (
194             "time-origin reset must be metric-invariant (score built from pure
195             differences)")
196
197         # (2) Absolute timeline differs ? the reset pulls control_segments toward 0.
198         seg_on = with_reset["control_segments"]
199         seg_off = without_reset["control_segments"]
200         assert len(seg_on) == len(seg_off) and len(seg_on) >= 1

```

```

197     assert seg_on[0][0] < seg_off[0][0], "reset must shift the first segment start
toward 0"
198     # The shift equals the offset in seconds (within rounding).
199     shift = seg_off[0][0] - seg_on[0][0]
200     assert abs(shift - offset_frames / fps) < 1e-3, (
201         f"segment shift {shift} should equal the frame offset in seconds {offset_frames
/ fps}")
202     # Reset trajectory starts at frame 0.
203     first_frame = int((tmp_path / "with" / "fx_reset_on" / "trajectory.csv")
204         .read_text(encoding="utf-8").splitlines()[1].split(",")[0])
205     assert first_frame == 0, "with reset, the trajectory must start at Frame 0"
206
207
208     # -----
209     # P1 (e/f) ? round-trip / correct-by-construction
210     # -----
211
212
213     def test_freeze_real_round_trips_check(tmp_path):
214         """After freeze_real_fixture, replay_fixture + compare_expected vs the written
expected.json
215         returns [] ? idempotent / correct-by-construction (the same path M1's --check
uses)."""
216         fixtures = tmp_path / "fixtures"
217         traj, labels, fps, fw, _ = _write_source(tmp_path)
218         expected = gf.freeze_real_fixture(
219             "scenario", traj, labels, slug="fx_rt000001",
220             license="owned", minors_free=True, owned=True,
221             fps=fps, frame_width=fw, fixtures_dir=fixtures)
222
223         fdir = fixtures / expected["slug"]
224         committed = gf._read_json(fdir / "expected.json")
225         recomputed = gf.replay_fixture(fdir)
226         assert gf.compare_expected(recomputed, committed) == [], (
227             "a just-frozen real fixture must replay-compare clean (correct-by-
construction)")
228
229
230     def test_freeze_real_uses_strict_label_loader(tmp_path):
231         """The labels are read through the STRICT gt_loader ? a malformed label row fails
loudly,
232         never silently shrinks the GT set."""
233         fixtures = tmp_path / "fixtures"
234         traj, labels, fps, fw, _ = _write_source(tmp_path)
235         # Corrupt one label row (end <= start) ? strict loader must raise.
236         labels.write_text("start,end\n5.0,1.0\n", encoding="utf-8", newline="\n")
237         with pytest.raises(ValueError):
238             gf.freeze_real_fixture(
239                 "scenario", traj, labels, slug="fx_strict01",
240                 license="owned", minors_free=True, owned=True,
241                 fps=fps, frame_width=fw, fixtures_dir=fixtures)
242

```

8. assistant (2026-06-16T06:27:13.190Z)

Now let me read the core dependencies that the replay reuses ? the algorithm that windowing changes would touch.

9. user (2026-06-16T06:27:14.152Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across

```

```

6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9         WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10         filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11         active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12         -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21         python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37     from backend.eval import windowing as _windowing
38
39     Interval = Tuple[float, float]
40
41     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
42     # Recall-oriented proposal: deliberately permissive so real rallies are among the
43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
44     # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
that
45     # module's docstring for the eval?serve bug these named presets close. The bare tuples
are
46     # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
47     # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
48     PROPOSAL = _windowing.PROPOSAL.as_tuple()
49     BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
learning
50
51
52     def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
53         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
54         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
55         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
56         vt, mg, mwd = proposal
57         wins = TrackNetRunner.trajectory_to_action_windows(
58             points, fps=fps, frame_width=frame_width,
59             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
60         intervals = [(w["start"], w["end"]) for w in wins]
61         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
62         X: List[List[float]] = []

```

```

63         for w, g in zip(wins, gated):
64             dur = max(1e-6, w["end"] - w["start"])
65             win_frames = max(1.0, dur * fps)
66             X.append([
67                 dur,                                     # rally length (sec) ?
68                 float(w["peak_velocity"]),               # already normalized by
69                 float(w["mean_velocity"]),
70                 float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
71                 float(g.crossings),                     # net crossings (count) ?
72             ])
73         return intervals, X
74
75
76     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
77         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
78         vt, mg, mwd = BASELINE_LOCAL
79         wins = TrackNetRunner.trajectory_to_action_windows(
80             points, fps=fps, frame_width=frame_width,
81             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82         return [(w["start"], w["end"]) for w in wins]
83
84
85     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86         fps, fw = float(spec["fps"]), float(spec["frame_width"])
87         intervals, X = build_candidates(spec["trajectory"], fps, fw)
88         gts = load_golden_gts(spec["labels"])
89         y = label_candidates(intervals, gts, iou)
90         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
91                 "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
92
93
94     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
95                         threshold: float = 0.5) -> List[Interval]:
96         if not intervals:
97             return []
98         proba = model.predict_proba(X)
99         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
100        return merge_overlaps(pos, gap_tol=1.0)
101
102
103     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
104             model_out: Optional[str] = None) -> dict:
105         videos = [load_video(s, iou) for s in specs]
106         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
107               f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
108         for v in videos:
109             ceil = score(v["intervals"], v["gts"], iou).recall
110             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
111                   f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
112
113         result: dict = {"videos": [v["name"] for v in videos]}
114         if len(videos) >= 2:
115             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
116             clf, base = [], []
117             for i, held in enumerate(videos):
118                 others = [v for j, v in enumerate(videos) if j != i]

```

```

119         X = [row for v in others for row in v["X"]]
120         y = [t for v in others for t in v["y"]]
121         model = LogisticRegression().fit(X, y, FEATURE_NAMES)
122         preds = predict_rallies(model, held["X"], held["intervals"], threshold)
123         sc = score(preds, held["gts"], iou)
124         bsc = score(held["baseline"], held["gts"], iou)
125         clf.append(sc)
126         base.append(bsc)
127         print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}"
128             f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
129     def mean(xs, a):
130         return sum(getattr(s, a) for s in xs) / len(xs)
131     print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
132         f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
133         f"F1={mean(base, 'f1'):.3f} R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
134     result["mean_learned_f1"] = mean(clf, "f1")
135     result["mean_baseline_f1"] = mean(base, "f1")
136
137     # Final model on ALL videos (for production / running on a fresh held-out video).
138     Xall = [row for v in videos for row in v["X"]]
139     yall = [t for v in videos for t in v["y"]]
140     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141     assert final.w is not None # set by fit()
142     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143     print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144     if model_out:
145         final.save(model_out)
146         print(f"final model (all videos) -> {model_out}")
147         result["model_out"] = model_out
148     return result
149
150
151 def main() -> None:
152     ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153     ap.add_argument("--manifest", required=True,
154                     help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155     ap.add_argument("--iou", type=float, default=0.5)
156     ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157     ap.add_argument("--model-out", default="output/local_rally_model.json")
158     args = ap.parse_args()
159     with open(args.manifest, encoding="utf-8") as f:
160         specs = json.load(f)
161     run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164 if __name__ == "__main__":
165     main()
166

```

10. user (2026-06-16T06:27:14.986Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6     1. Translate the Windows video path to its WSL-visible form (and optionally
7       stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8     2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.

```

```

9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14    Nothing here imports TensorFlow; all model code stays in WSL.
15    """
16
17    import os
18    import csv
19    import shlex
20    import logging
21    import subprocess
22    from dataclasses import dataclass
23    from typing import List, Dict, Any, Optional, Tuple
24
25    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27    logger = logging.getLogger(__name__)
28
29
30    @dataclass
31    class TrackNetConfig:
32        """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33        distro: str = "Ubuntu"
34        repo_dir: str = "~/models/TrackNetV4" # WSL path to the cloned repo
35        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36        conda_env: str = "TrackNetV4"
37        weights_path: str = "" # WSL path to the .h5/.keras weights
(REQUIRED)
38        queue_length: int = 5
39        stage_in_wsl: bool = True # copy video into WSL fs first
(perf)
40        wsl_stage_dir: str = "~/clips" # where staged videos/outputs live
in WSL
41        timeout_sec: int = 1800 # 30 min; kills a hung call (0 = no
timeout)
42        keep_frames: bool = False # retain staged video after success
(debug only)
43        min_free_gb: float = 0.0 # warn if WSL free space below this
(0 = off)
44
45    @classmethod
46    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47        tn = (idx_cfg or {}).get("tracknet", {}) or {}
48        return cls(
49            distro=tn.get("wsl_distro", "Ubuntu"),
50            repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51            conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52            conda_env=tn.get("conda_env", "TrackNetV4"),
53            weights_path=tn.get("weights_path", ""),
54            queue_length=int(tn.get("queue_length", 5)),
55            stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56            wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57            timeout_sec=int(tn.get("timeout_sec", 1800)),
58            keep_frames=bool(tn.get("keep_frames", False)),
59            min_free_gb=float(tn.get("min_free_gb", 0.0)),
60        )
61
62
63    def to_wsl_mnt_path(win_path: str) -> str:
64        """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65

```

```

66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "`wsl` not found ? is this running on Windows with WSL2 installed?"
112         except subprocess.TimeoutExpired:
113             return False, "WSL healthcheck timed out."
114         out = (res.stdout or "").strip()
115         if res.returncode != 0:
116             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
117         return True, out
118
119 # ----- inference
120
121 def run_predict(self, video_win_path: str, output_win_dir: str,
122                 video_id: Optional[str] = None) -> Optional[str]:
123     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
124     None.
125
126     ``output_win_dir`` is a Windows directory (under the repo's output/) that
127     WSL writes into via its /mnt view, so the Windows process can read the CSV
128     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
129     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share

```

```

128         a filename cannot collide on the output CSV.
129         ""
130     if not self.cfg.weights_path:
131         logger.error(
132             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
133             "in config.json to the WSL path of your trained TrackNetV4 weights."
134         )
135     return None
136
137     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138     # relative paths through unchanged, so WSL would resolve them against the
139     # predict.py cwd instead of the repo (same bug class as wasb_runner).
140     output_win_dir = os.path.abspath(output_win_dir)
141     os.makedirs(output_win_dir, exist_ok=True)
142     # Normalize separators so basename/splitext work when tests (or collector)
143     # pass windows-style paths on a Linux host.
144     video_win_path = str(video_win_path).replace("\\", "/")
145     base = os.path.basename(video_win_path)
146     stem, ext = os.path.splitext(base)
147
148     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149     if ext.lower() not in (".mp4", ".avi"):
150         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151     return None
152
153     wsl_out = to_wsl_mnt_path(output_win_dir)
154     # Namespace by video_id so two same-filename videos don't collide on the CSV.
155     # predict.py derives its output name from the (staged) video stem, so staging
156     # under the namespaced name propagates into "<key>_predict.csv".
157     key = f"{stem}_{video_id[:12]}" if video_id else stem
158     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160     # Resolve the video path WSL will read.
161     if self.cfg.stage_in_wsl:
162         staged = self._stage_video(video_win_path, f"{key}{ext}")
163         if staged is None:
164             return None
165         wsl_video = staged
166     else:
167         # No staging: predict.py reads /mnt directly and writes
168         # "<stem>_predict.csv".
169         # We cannot rename its output, so fall back to the un-namespaced name.
170         wsl_video = to_wsl_mnt_path(video_win_path)
171         expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173     if self.cfg.min_free_gb > 0:
174         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175         if free is not None and free < self.cfg.min_free_gb:
176             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177                             "consider running tools/cleanup_caches.py.",
178                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180     cmd = (
181         f"conda activate {self.cfg.conda_env} && "
182         f"cd {self.cfg.repo_dir}/src && "
183         f"python predict.py "
184         f"--video_path {wsl_tilde_quote(wsl_video)} "
185         f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186         f"--output_dir {wsl_tilde_quote(wsl_out)} "
187         f"--queue_length {self.cfg.queue_length}"
188     )
189     logger.info("Running TrackNet inference on %s ...", base)
190     try:
191         res = self._wsl_bash(cmd)
192     except subprocess.TimeoutExpired:

```

```

192         logger.error("TrackNet inference timed out after %ss.",
self.cfg.timeout_sec)
193         return None
194
195     if res.returncode != 0:
196         logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
res.stdout)[-2000:])
197         return None
198
199     if not os.path.exists(expected_csv_win):
200         logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201         return None
202         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204     # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205     # regenerable intermediate). On earlier failure we return above without
cleaning.
206     if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207         logger.info("Cache hygiene: removing staged video for %s "
208                     "(set indexing.tracknet.keep_frames=true to retain).", key)
209         self._wsl_rm_rf([wsl_video])
210     return expected_csv_win
211
212 def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213     """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215     Returns the WSL path to the staged copy, or None on failure.
216     """
217     src_mnt = to_wsl_mnt_path(video_win_path)
218     reason = self.wsl_source_unreadable_reason(src_mnt)
219     if reason:
220         logger.error("Cannot stage video into WSL: %s", reason)
221         return None
222     dest = f"{self.cfg.wsl_stage_dir}/{base}"
223     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224     try:
225         res = self._wsl_bash(cmd)
226     except subprocess.TimeoutExpired:
227         logger.error("Staging video into WSL timed out.")
228         return None
229     if res.returncode != 0 or "OK" not in (res.stdout or ""):
230         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231         return None
232     return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):

```

```

251             continue
252         points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,
259         frame_width: float,
260         chunk_start: float = 0.0,
261         velocity_thresh: float = 0.02,
262         merge_gap: float = 2.0,
263         min_window_duration: float = 2.0,
264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267     ) -> List[Dict[str, Any]]:
268         """Convert a shuttle trajectory into candidate rally windows.
269
270         A frame is "active" when the shuttle is visible AND its inter-frame
271         displacement (normalised by frame width, per second) exceeds
272         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
273         ``merge_gap`` seconds of each other are grouped into one window; short
274         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
275         shape feeding the AI-handoff phase is identical.
276
277         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
278         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
279         progress" and a single window swallows several rallies + the dead time between them. When
280         set, any window longer than this is recursively SPLIT at its largest internal inactivity
281         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
282         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
283         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
284         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
285         """
286         if fps <= 0:
287             fps = 30.0
288         if frame_width <= 0:
289             frame_width = 1280.0
290
291         # Compute per-frame normalised velocity from the last *visible* point.
292         active = [] # (frame_idx, velocity)
293         prev: Optional[TrajectoryPoint] = None
294         for p in points:
295             if not p.visible:
296                 prev = None # break the trajectory; absent shuttle = not in flight
297                 continue
298             if prev is not None:
299                 dframes = p.frame - prev.frame
300                 if dframes > 0:
301                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
302                     velocity = (dist / frame_width) * (fps / dframes)
303                     if velocity > velocity_thresh:
304                         active.append((p.frame, velocity))
305                 prev = p
306
307         if not active:
308             return []

```

```

309
310     max_gap_frames = int(merge_gap * fps)
311
312     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
313         vels = [v for _, v in group]
314         return {
315             "start": group[0][0] / fps + chunk_start,
316             "end": group[-1][0] / fps + chunk_start,
317             "active_frame_count": len(group),
318             "peak_velocity": max(vels),
319             "mean_velocity": sum(vels) / len(vels),
320             "track_id_count": 1, # single shuttle; kept for window-shape parity
321             "chunk_index": chunk_index,
322         }
323
324     groups: List[List[Tuple[int, float]]] = []
325     group = [active[0]]
326     for af in active[1:]:
327         if af[0] - group[-1][0] <= max_gap_frames:
328             group.append(af)
329         else:
330             groups.append(group)
331             group = [af]
332     groups.append(group)
333
334     if max_window_duration > 0:
335         groups = TrackNetRunner._split_overlong_groups(
336             groups, int(max_window_duration * fps), int(min_split_gap * fps))
337
338     windows = [_finalize(g) for g in groups]
339     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
340
341     @staticmethod
342     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
343     int,
344                                min_split_frames: int) -> List[List[Tuple[int, float]]]:
345         """Recursively split any active-frame group longer than ``max_win_frames`` at
346         its largest
347         likely rally
348         boundary. Splitting only subdivides existing groups (never merges/extends), so
349         it cannot
350         invent or drop coverage; a group with no gap ? the floor is left intact (a
351         genuinely long
352         rally). Iterative worklist to bound stack depth."""
353         if max_win_frames <= 0:
354             return groups
355         out: List[List[Tuple[int, float]]] = []
356         work = list(groups)
357         while work:
358             g = work.pop()
359             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
360                 out.append(g)
361                 continue
362             best_i, best_gap = -1, -1
363             for i in range(1, len(g)):
364                 gap = g[i][0] - g[i - 1][0]
365                 if gap > best_gap:
366                     best_gap, best_i = gap, i
367             if best_i < 0 or best_gap < min_split_frames:
368                 out.append(g) # too long but no real dead-time gap ? keep as one rally
369                 continue
370             work.append(g[:best_i])
371             work.append(g[best_i:])
372         out.sort(key=lambda grp: grp[0][0])

```

```
369         return out
370
```

11. assistant (2026-06-16T06:27:18.179Z)

Now let me read the rally_gate, experiment, metrics, and gt_loader files.

12. user (2026-06-16T06:27:19.613Z)

```
1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4      the opponent for service after losing a point, that single trajectory looks like a
5      rally. The discriminator is structural, not duration-based: a real rally has the
6      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7      single service feed crosses ~once.
8
9      This module is camera-agnostic and needs NO new model ? it works on the trajectory
10     we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11     axis with the larger shuttle spread) and the net position (median shuttle coord on
12     that axis ? the shuttle clusters at the two ends and crosses the net region often,
13     so the median sits near the net), then counts sign changes of (coord - net) across
14     the visible points inside a candidate window, with a deadband to ignore jitter.
15
16     Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17     Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18     windows from trajectory_to_action_windows, or to fold into it later.
19     """
20     from __future__ import annotations
21
22     from dataclasses import dataclass
23     from typing import List, Optional, Sequence, Tuple
24
25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
47     def _median(vals: Sequence[float]) -> float:
48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
56         """Return (axis 'x'|'y', net_value, span) from visible points.
```

```

57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:
105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113 def apply_gate(points, windows: List[Interval], fps: float,
114               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115     """Convenience: return only the windows that pass the crossing gate."""
116     return [(g.start, g.end) for g in
117             gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

13. user (2026-06-16T06:27:20.496Z)

```
1      """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3      This is the thin variant/experiment layer that `docs/EXPERIMENT_HARNESS.md`
4      identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5      no-regression gate, the pinnable classifier artifact, the persisted-candidate
6      substrate) already exists ? this module composes it. No new scoring math lives
7      here; everything defers to:
8
9      - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10     - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11     - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12     - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13     - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14     - `backend.eval.calibration`   ? `pct_improvement`
15
16     The thesis (`EXPERIMENT_HARNESS.md` §2): separate the expensive, risky DETECTION
17     (per-frame signals ? run once on the GPU/WSL box, persisted into
18     `candidate_segments.stats`) from the cheap, swappable DECISION (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); rollback = flip the variant/config, never `git revert`.
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
56     ratio
57     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
58     scores
59
60     logger = logging.getLogger(__name__)
61
62     # Where a comparison run appends one reproducible record. Tracked location (NOT under
63     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
```

```

62 # regression.DEFAULT_BASELINE_PATH.
63 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64 _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67 class ExperimentError(ValueError):
68     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69     score an unlabeled video with no pinned model, or a gate referencing an absent
70     feature)."""
71
72
73 # ----- Variant
74
75
76 @dataclass
77 class Variant:
78     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80     A variant turns one video's persisted candidate windows + features into rally
81     intervals. Every knob defaults to the control behaviour (no gate, no learned
82     filter), so a variant that merely carries a new, disabled cue is byte-identical
83     to control ? the core no-regression guarantee.
84
85     Fields:
86     - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87       for the leaderboard and for selecting the served path (the existing
88       ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89       New cues are recorded here default-OFF.
90     - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91       ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92       ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93       windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94     - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95       whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96       cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98       ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99       required for ``compare_unlabeled`` on a learned variant.
100     - ``feature_names`` ? the persisted features the learned filter consumes. When set
101       WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102       (honest) ? the recipe, not a pinned artifact.
103     - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104       overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105     """
106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0
112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
122

```

```

123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:

```

```

186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                         name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196         cols = [_feature_column(vc, name) for name in feature_names]
197         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202         players)."""
203         n = len(vc.intervals)
204         mask = [True] * n
205         if variant.min_crossings > 0:
206             col = _feature_column(vc, "net_crossings")
207             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208         if variant.min_players > 0:
209             col = _feature_column(vc, "players_in_court")
210             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211         return mask
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted ``LogisticRegression``) is supplied by ``compare`` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225         them.
226
227         """
228         mask = _gate_mask(variant, vc)
229         if variant.feature_names:
230             if model is None:
231                 if variant.classifier_model is None:
232                     raise ExperimentError(
233                         f"variant {variant.name!r} is learned but has no model to predict "
234                         f"with (pass a fitted model or set classifier_model)")
235                 model = LogisticRegression.load(variant.classifier_model)
236             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]
238             if calibrator is not None:
239                 proba = [calibrator(p) for p in proba]
240             thr = variant.threshold if threshold is None else threshold
241             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243             return merge_overlaps(kept, gap_tol=variant.merge_gap)
244
245     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
246                             train_videos: Sequence[VideoCandidates], iou: float
247                             ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":

```

```

246     """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247     (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248     (use the variant default). Honest: never looks at the held-out video.
249     """
250     calibrator: Optional[Callable[[float], float]] = None
251     if variant.calibrate:
252         raw: List[float] = []
253         y: List[int] = []
254         for vc in train_videos:
255             raw.extend(float(p) for p in
256                         model.predict_proba(_feature_matrix(vc, variant.feature_names or
257                                                         [])))
258             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
259         if variant.calibrate == "platt":
260             calibrator = fit_platt(raw, y)
261         elif variant.calibrate == "isotonic":
262             calibrator = fit_isotonic(raw, y)
263         else:
264             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
265                                   'platt'/'isotonic')")
266     threshold: Optional[float] = None
267     if variant.tune_threshold:
268         best_t, best_f1 = variant.threshold, -1.0
269         for k in range(1, 20):
270             # grid 0.05..0.95 on the train
271             t = round(0.05 * k, 2)
272             f1s = [score(predict(variant, vc, model=model, threshold=t,
273                                 calibrator=calibrator),
274                     vc.gts or [], iou).f1 for vc in train_videos]
275             mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
276             if mean_f1 > best_f1:
277                 best_f1, best_t = mean_f1, t
278             threshold = best_t
279     return calibrator, threshold
280
281 # ----- variant scoring
282
283 def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
284 LogisticRegression:
285     """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
286     the
287     evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
288     X: List[List[float]] = []
289     y: List[int] = []
290     for vc in videos:
291         if vc.gts is None:
292             raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
293         X.extend(_feature_matrix(vc, variant.feature_names or []))
294         y.extend(label_candidates(vc.intervals, vc.gts, iou))
295     if not X:
296         raise ExperimentError("cannot train a learned variant on zero candidates")
297     return LogisticRegression().fit(X, y, variant.feature_names)
298
299 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
300                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
301     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
302     mean aggregate.
303
304     Prediction policy:
305     - **static variant** (no learned filter): predict each video directly ? no
306     training,
307     so no leakage; per-video scoring is already honest.

```

```

304         - **learned, pinned model** (``classifier_model`` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
306         to
307         report "how the shipped artifact does here", not for the promotion decision.
308         - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
309         ?
310         train on the OTHER videos, predict the held-out one. The number to trust.
311         - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
312         only).
313         """
314         for vc in videos:
315             if vc.gts is None:
316                 raise ExperimentError(f"{vc.name} has no golden labels; use
317                 compare_unlabeled")
318             per_video: List[Dict[str, Any]] = []
319             predictions: Dict[str, List[Interval]] = {}
320             recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
321             pinned_model = (LogisticRegression.load(variant.classifier_model)
322                             if variant.classifier_model is not None else None)
323             all_model = None
324             if recipe_lovo and not honest:
325                 all_model = _train(variant, videos, iou)
326             for i, vc in enumerate(videos):
327                 cal: Optional[Callable[[float], float]] = None
328                 thr: Optional[float] = None
329                 if recipe_lovo and honest:
330                     others = [v for j, v in enumerate(videos) if j != i]
331                     if not others:
332                         raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
333                         pinned model")
334                     model: Optional[LogisticRegression] = _train(variant, others, iou)
335                     if variant.tune_threshold or variant.calibrate: # operating point from
336                         the train fold
337                             assert model is not None # _train never returns
338                             None
339                             cal, thr = _calibrate_and_tune(variant, model, others, iou)
340                     elif recipe_lovo: # honest=False ? one model over all
341                         videos
342                         model = all_model
343                     else: # static variant or pinned model
344                         model = pinned_model
345                         preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
346                         assert vc.gts is not None # guarded at function top
347                         sc = score(preds, vc.gts, iou)
348                         per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
349                         for m in _METRICS}})
350                         predictions[vc.name] = preds
351                     aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
352                     0.0)
353                     for m in _METRICS}
354             return {
355                 "variant": variant.name,
356                 "spec_hash": variant.spec_hash(),
357                 "is_learned": variant.is_learned(),
358                 "honest_lovo": recipe_lovo and honest,
359                 "per_video": per_video,
360                 "aggregate": aggregate,
361                 "predictions": predictions,
362             }

```

```

359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS $6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404               iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` $5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

```

```

411
412     ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413     paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414     existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415     shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416     bare LOVO mean at n?6 is not trustworthy on its own.
417     ""
418     if len(videos) < 1:
419         raise ExperimentError("compare needs at least one labeled video")
420     eval_a = evaluate_variant(videos, variant_a, iou, honest)
421     eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423     delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424     delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426     by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427     per_video_delta = [
428         {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429         for p in eval_b["per_video"]
430     ]
431
432     # One fingerprint over the whole label set ? detects "the deltas were measured
against
433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455         agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465             the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a

```

```

468 pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469 `metrics.match_intervals` ? no new matching logic.
470 """
471 preds_a = predict(variant_a, video)
472 preds_b = predict(variant_b, video)
473 mr = match_intervals(preds_a, preds_b, agree_iou)
474
475 agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476           for m in mr.matches]
477 agree_iou = [m.iou for m in mr.matches]
478 a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479 b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
480
481 def _dur(ivs: List[Interval]) -> float:
482     return sum(e - s for s, e in ivs)
483
484 return {
485     "video": video.name,
486     "agree_iou": agree_iou,
487     "variant_a": variant_a.name,
488     "variant_b": variant_b.name,
489     "num_a": len(preds_a),
490     "num_b": len(preds_b),
491     "num_agreed": len(agreed),
492     "mean_agree_iou": (sum(agree_iou) / len(agree_iou)) if agree_iou else 0.0,
493     "duration_a": _dur(preds_a),
494     "duration_b": _dur(preds_b),
495     "agreed": agreed,
496     "a_only": a_only,
497     "b_only": b_only,
498 }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:

```

```

532         entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538                 row["variant_a"]["name"], row["variant_b"]["name"],
539                 (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551         touching
552         function; the comparison itself is pure and offline. Exercised owner-side on a real
553         DB,
554         but works against any `Database` (including an in-memory one in tests).
555         """
556         rows = db.query_candidate_segments(video_id, detector=detector)
557         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558                                     rows]
559         features: Dict[str, List[float]] = {}
560         for name in feature_names:
561             if name == "duration":
562                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
563                                 rows]
564             else:
565                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
566                                 rows]
567         return VideoCandidates(name=video_id, intervals=intervals, features=features,
568                                gts=gts)
569

```

14. assistant (2026-06-16T06:27:23.452Z)

Now let me read metrics.py, gt_loader.py, regression.py, and the windowing module to understand the SERVED constant and how the literal vs SERVED divergence works.

15. user (2026-06-16T06:27:24.888Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is

```



```

82
83
84 @dataclass
85 class Score:
86     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87     iou_threshold: float
88     num_pred: int
89     num_gt: int
90     true_positives: int
91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float          # over matched pairs (0.0 if none)
97     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104 def score(preds: List[Interval], gts: List[Interval],
105          iou_threshold: float = 0.5,
106          match_result: Optional[MatchResult] = None) -> Score:
107     """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109     Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110     extracting the failure lists); otherwise matching is done here.
111     """
112     mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114
115     tp = len(mr.matches)
116     fp = len(mr.unmatched_pred_indices)
117     fn = len(mr.unmatched_gt_indices)
118
119     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122 else 0.0
123
124     if mr.matches:
125         mean_iou = sum(m.iou for m in mr.matches) / tp
126         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127 mr.matches) / tp
128         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129 mr.matches) / tp
130     else:
131         mean_iou = mean_start_err = mean_end_err = 0.0
132
133     return Score(
134         iou_threshold=iou_threshold,
135         num_pred=len(preds), num_gt=len(gts),
136         true_positives=tp, false_positives=fp, false_negatives=fn,
137         precision=precision, recall=recall, f1=f1,
138         mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
139     )
140
141
142 def pr_curve(preds: List[Interval], gts: List[Interval],
143             thresholds: Optional[List[float]] = None) -> List[Score]:
144     """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
145
146     Lets you see strict-vs-lenient behaviour rather than a single operating point.

```

```

143     """
144     if thresholds is None:
145         thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146     return [score(preds, gts, iou_threshold=t) for t in thresholds]
147

```

16. user (2026-06-16T06:27:25.380Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20    """
21    import csv
22    import logging
23
24    from typing import List, Optional, Tuple
25
26    from backend.eval.annotations import parse_timestamp
27
28    logger = logging.getLogger(__name__)
29
30    Interval = Tuple[float, float]
31
32    _START_NAMES = ("start", "start_time")
33    _END_NAMES = ("end", "end_time")
34
35    def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
36        """If `row` is a header naming start/end columns, return their indices, else
37        None."""
38        cells = [(c or "").strip().lower() for c in row]
39        start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
40        end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
41        if start_idx is not None and end_idx is not None:
42            return start_idx, end_idx
43        return None
44
45    def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
46        """Load GT rally intervals from either golden-CSV convention.
47
48        strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
49        (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
50        strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
51        skipped ? a silently shrinking GT set flatters precision.
52
53        Returns intervals sorted by start; overlapping GT intervals are logged as a warning
54        (they break one-to-one matching and usually indicate a labeling error).
55        """

```

```

55     intervals: List[Interval] = []
56     skipped = 0
57     cols: Optional[Tuple[int, int]] = None    # (start_idx, end_idx); None until detected
58     saw_header = False
59
60     with open(csv_path, newline="", encoding="utf-8-sig") as f:
61         for lineno, row in enumerate(csv.reader(f), start=1):
62             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                 continue
64             if cols is None and not saw_header:
65                 header = _header_columns(row)
66                 if header is not None:
67                     cols = header
68                     saw_header = True
69                 continue                # consume the header row
70             cols = (0, 1)                # headerless: positional start,end
71             assert cols is not None       # every branch above assigns it
72             si, ei = cols
73             try:
74                 if len(row) <= max(si, ei):
75                     raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                 start = parse_timestamp(row[si])
77                 end = parse_timestamp(row[ei])
78                 if start < 0:
79                     raise ValueError(f"negative start ({start})")
80                 if start >= end:
81                     raise ValueError(f"start ({start}) >= end ({end})")
82             except ValueError as e:
83                 if strict:
84                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                 skipped += 1
86                 continue
87             intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

17. user (2026-06-16T06:27:26.121Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7         ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8         predictions  ? the production pipeline's stored DB rows    [backend.eval.harness]
9         comparison   ? interval matching + P/R/F1                  [backend.eval.metrics]
10        baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.

```

```

16     """
17     import os
18     import json
19     import hashlib
20     import logging
21     from dataclasses import dataclass, asdict
22     from typing import Dict, List, Optional
23
24     from backend.infrastructure.database import Database
25     from backend.eval import metrics
26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78

```

```

79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                       f1: float, path: str = DEFAULT_BASELINE_PATH,
82                       iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                       gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100                    _baseline_key(video_id, stage), precision, recall)
101
102     # ----- the runner
103
104     def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105                stage: str = "final", iou_threshold: float = 0.5,
106                detector: Optional[str] = None,
107                tolerance: float = DEFAULT_TOLERANCE,
108                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109         """Score stored predictions for one video against ground truth and compare to
110         baseline.
111
112         `ground_truth` is a list of (start, end) intervals ? typically obtained from an
113         AnnotationProvider (see `regress_with_provider`). A regression is flagged when
114         precision OR recall falls more than `tolerance` below the snapshotted baseline.
115         If no baseline exists yet, `regressed` is False (nothing to compare to) and the
116         caller can choose to snapshot the current numbers via `save_baseline`.
117         """
118         preds = get_predictions(db, video_id, stage, detector=detector)
119         s = metrics.score(preds, ground_truth, iou_threshold)
120         fp = gt_hash(ground_truth)
121
122         baselines = load_baselines(baseline_path)
123         b = baselines.get(_baseline_key(video_id, stage))
124
125         reasons: List[str] = []
126         regressed = False
127         has_baseline = b is not None
128         base_p = base_r = None
129         if b is not None:
130             base_p, base_r = b["precision"], b["recall"]
131             # Incommensurable comparison guard: a baseline taken at a different IoU/detector
132             or
133             # against different labels is not a valid reference ? flag it instead of
134             trusting it.
135             b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
136             b.get("gt_hash")
137             if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
138                 regressed = True
139             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
140             (incommensurable)")
141             if b_det is not None and b_det != detector:

```

```

137         regressed = True
138         reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
157         )
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                             tier: str = "file", stage: str = "final",
162                             iou_threshold: float = 0.5,
163                             detector: Optional[str] = None,
164                             tolerance: float = DEFAULT_TOLERANCE,
165                             baseline_path: str = DEFAULT_BASELINE_PATH,
166                             guideline: Optional[str] = None,
167                             sport: str = "badminton",
168                             provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + "\n    - {why}" for why in r.reasons)
189

```

18. assistant (2026-06-16T06:27:29.563Z)

Now let me read the windowing module (the SERVED constant), the CI workflow, the docs, and the committed fixtures.

19. user (2026-06-16T06:27:30.870Z)

```
1      """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3      **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4      ?
5      ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
6      built
7      on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8      deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9      pipeline
10     (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11     merge_gap=2.0,
12     min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13     is
14     measured on candidates **production never serves**, so a "win" need not transfer. This
15     is
16     exactly how Gate-A scored **0.074** on the proposal set yet **0.008** on the served
17     set
18     (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20     **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
21     the
22     SERVED preset is **single-sourced from the same Pydantic config defaults production
23     reads**
24     (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
25     Helpers
26     build candidates under either preset, and :func:`served_windowing_from_config` lifts the
27     *actual* served windowing out of a live config (overrides included), so an A/B can
28     target the
29     operating point a given deployment truly serves.
30
31     The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
32     windowing* ? lives in :mod:`backend.eval.promotion` (which composes `eval_stats` +
33     `per_user_gate`). This module is the **what windowing** ; that one is the **promotion
34     gate**.
35
36     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
37     a
38     caller opts in (`distill_local` re-points its module-level presets here, byte-
39     identically).
40
41     """
42     from __future__ import annotations
43
44     from dataclasses import dataclass
45     from typing import Any, Dict, List, Mapping, Tuple
46
47     from backend.config.models import IndexingConfig
48
49     Interval = Tuple[float, float]
50
51     __all__ = [
52         "WindowingPreset",
53         "SERVED",
54         "PROPOSAL",
55         "PRESETS",
56         "served_windowing_from_config",
57         "build_windows",
58         "windows_to_intervals",
59     ]
60
61     @dataclass(frozen=True)
62     class WindowingPreset:
63         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
```

```

point.
50
51     ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52     "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53     ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54     knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55     ""
56
57     name: str
58     velocity_thresh: float
59     merge_gap: float
60     min_window_duration: float
61     #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62     #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63     max_window_duration: float = 0.0
64
65     def as_tuple(self) -> Tuple[float, float, float]:
66         """"(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
67         ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
68         bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)""
69         return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
70
71     def to_dict(self) -> Dict[str, Any]:
72         return {
73             "name": self.name,
74             "velocity_thresh": self.velocity_thresh,
75             "merge_gap": self.merge_gap,
76             "min_window_duration": self.min_window_duration,
77             "max_window_duration": self.max_window_duration,
78         }
79
80
81     def _served_preset_from_defaults() -> WindowingPreset:
82         """"The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
83
84         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
85         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
86         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
87         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
88
89         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
90         point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
91         so the two can never silently diverge again.
92         ""
93         idx = IndexingConfig() # construct with defaults ? the served operating point
94         return WindowingPreset(
95             name="served",
96             velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
97             merge_gap=float(idx.dead_time_merge_gap),
98             min_window_duration=float(idx.min_action_window_duration),
99             max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing

```

```

(default 0=OFF)
100     )
101
102
103     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
104     #: only promotable if it holds up HERE ? the operating point real users get.
105     SERVED: WindowingPreset = _served_preset_from_defaults()
106
107     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
108     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
109     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
110     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
111     PROPOSAL: WindowingPreset = WindowingPreset(
112         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
113     )
114
115     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
116     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
117
118
119     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
120         """The SERVED windowing a *specific* live config serves (overrides honoured).
121
122         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
123         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
124         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
125         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
126         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
127         """
128         idx = _as_mapping(config, "indexing")
129         family_cfg = _as_mapping(idx, family)
130         return WindowingPreset(
131             name=f"served:{family}",
132             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
133             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
134             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
135             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
136         )
137
138
139     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
140         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
141         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
142         if obj is None:
143             return {}
144         if isinstance(obj, Mapping):
145             sub = obj.get(key, {})
146         elif hasattr(obj, "get"):
147             sub = obj.get(key, {})
148         else:
149             sub = getattr(obj, key, {})

```

```

150         return sub if isinstance(sub, Mapping) else {}
151
152
153     def build_windows(points: List[Any], fps: float, frame_width: float,
154                       preset: WindowingPreset) -> List[Dict[str, Any]]:
155         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
156
157         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
158         instead of three loose floats, so eval candidate-building and the served path
reference the
159         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
160         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
161
162         vt, mg, mwd = preset.as_tuple()
163         return TrackNetRunner.trajectory_to_action_windows(
164             points, fps=fps, frame_width=frame_width,
165             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
166             max_window_duration=preset.max_window_duration)
167
168
169     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
170         """``[[start, end, ...]]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
171         return [(w["start"], w["end"]) for w in windows]
172

```

20. user (2026-06-16T06:27:31.102Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11     concurrency:
12         group: ci-${{ github.ref }}
13         cancel-in-progress: true
14
15     jobs:
16         # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17         # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18         # tracked .py, including scripts/ and training/, which the backend-only
19         # py_compile smoke sweep does not reach.
20         lint:
21             name: Ruff lint
22             runs-on: ubuntu-latest
23             steps:
24                 - uses: actions/checkout@v6
25                 - uses: actions/setup-python@v6
26                   with:
27                       python-version: "3.13"
28                 - name: Install ruff
29                   run: python -m pip install ruff
30                 - name: Ruff check
31                   run: ruff check . --output-format github
32
33         # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34         # per-module quarantine list ? ratchet by removing entries as modules clean

```

```

35     # up. Typed core deps are installed so mypy sees their real signatures
36     # (without them ignore_missing_imports would Any-stub half the checks).
37     typecheck:
38         name: Mypy (lenient baseline)
39         runs-on: ubuntu-latest
40         steps:
41             - uses: actions/checkout@v6
42             - uses: actions/setup-python@v6
43               with:
44                 python-version: "3.13"
45             - name: Install mypy + typed core deps
46               run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47             - name: Mypy
48               run: mypy backend training
49
50     # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
51     # module-level import error shipping while the suite stays green.
52     # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53     # on machines without the GPU stack (test_import_smoke stubs what's absent).
54     import-smoke:
55         name: Syntax sweep + import smoke (no GPU stack)
56         runs-on: ubuntu-latest
57         steps:
58             - uses: actions/checkout@v6
59             - uses: actions/setup-python@v6
60               with:
61                 python-version: "3.13"
62             - name: Install minimal deps
63               run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64             - name: Run import smoke tests
65               run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68         name: Full test suite (offline, mocked)
69         runs-on: ubuntu-latest
70         steps:
71             - uses: actions/checkout@v6
72             - uses: actions/setup-python@v6
73               with:
74                 python-version: "3.13"
75                 cache: pip
76             - name: System libs for opencv-python
77               run: |
78                 sudo apt-get update
79                 sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
80             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81               # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82               # need an out-of-band index that PEP 621 can't express. Keep installing
83               # them separately here, before the editable install pulls the rest.
84               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85             - name: Install package (editable) + dev tooling
86               # Editable PEP 621 install: runtime deps + the `dev` group
87               # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88               run: python -m pip install -e .[dev]
89             # obsreport (private, soft dep) is absent here: reporting tests skip via
90             # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91             # is therefore calibrated against THIS lane's number (local runs with
92             # obsreport installed measure a little higher).
93             # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94             # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95             # without an owner decision.

```

```

96         - name: Run full suite (with coverage floor)
97         run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99         # Cross-OS determinism guard for the committed golden regression fixtures
100        # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101        # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102        # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103        # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104        # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105        golden-crossos:
106          name: Golden fixtures cross-OS (${{ matrix.os }})
107          strategy:
108            fail-fast: false
109          matrix:
110            os: [ubuntu-latest, windows-latest, macos-latest]
111          runs-on: ${{ matrix.os }}
112          steps:
113            - uses: actions/checkout@v6
114            - uses: actions/setup-python@v6
115              with:
116                python-version: "3.13"
117            - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
119            - name: Golden-fixtures characterization (cross-OS determinism guard)
120              run: |
121                python -m backend.eval.golden_fixtures --check
122                python -m pytest tests/test_golden_fixtures.py -q
123

```

21. assistant (2026-06-16T06:27:34.553Z)

Now let me read the docs (the claims §4c/§5/§6) and the committed fixtures + manifest.

22. user (2026-06-16T06:27:35.911Z)

```

1      # Golden Regression Fixtures ? video-free, non-attributable regression suite
2
3      > **Status:** `IN PROGRESS` ? owner-gated . **Owner:** Avi . **Created:** 2026-06-16 .
**Last updated:** 2026-06-16 . **Legal:** owner **accepted the researched risk** 2026-06-16
(conditions §3 stand; this is the owner's informed acceptance, **not** a retained-counsel
opinion)
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE, per archive policy)
5      > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
6      > **Relation to existing docs:** EXTENDS (does not supersede)
`docs/GOLDEN_DATA_SHARING.md` (#175); reuses the eval stack documented in `docs/EVALUATION.md` /
`docs/EXPERIMENT_HARNESS.md`; quality floor ties to `docs/QUALITY_ITERATIONS.md`.
7      > **Honesty note:** §3 (Legal) is **researched general information, NOT legal advice** ?
it gates on retained counsel (§8). §2/§4/§7 mark which claims are **[verified]** against code vs
**[design]** proposals.
8
9      ---
10
11     ## 0. TL;DR
12
13     Let a contributor on **another machine with no video, GPU, WSL, or DB** run the rally-
segmentation regression suite from **committed structured files**, so refactors and non-quality
PRs get a red/green side-effect signal. Two layers, built once and grown by accretion:
14
15     1. **Mechanics** ? freeze the deterministic pipeline at the **post-detector trajectory
seam** and replay it through the *already-shipped* eval stack (`experiment.py` / `metrics.py` /
`windowing.py` / `regression.py`), with two auto-discovered gates: a

```

****characterization/snapshot**** gate (catches any behaviour change) and a ****quality-floor**** gate (catches quality drops).

16 2. ****Privacy/legal filter (SEALED-FIXTURES)**** ? only ****de-attributed, minimized, owner-source, biometric-free**** numbers are ever committed; richer/sensitive streams stay key-gated or out-of-repo. Non-attributability comes from ****deletion-at-source****, not encryption.

17

18 ****The convergence that makes this clean:**** the **privacy** answer (publish shuttle-only, drop person/pose streams) and the **determinism** answer (freeze only the pure-stdlib ``CONTROL`` path, keep the numpy/BLAS learned scorer off the cross-machine gate) point at the ****same minimal public tier****. That coincidence is the spine of the design.

19

20 ****Two findings already actionable, independent of the rest:****

21 - ? ****Live de-attribution bug**** **(verified)**: real player/venue names are committed today in ``eval_baselines/heuristic_lovo_n6.json``, ``eval_baselines/gemini_wrapper_2026-06-10.json``, and ``backend/eval/eval_partial_labels.py`` (the last also has a personal ``C:/Users/avidu/...`` path) ? and in git history. ? ****Deliverable P0.****

22 - ? ****Determinism trap**** **(verified, corroborated by two independent red-teams)**: the learned variants (``shuttle_only``, ``fusion``) train a numpy/OpenBLAS ``DYNAMIC_ARCH`` logistic regression whose decisions flip near the 0.5 threshold across CPUs ? would false-RED on a contributor's machine. ? the cross-machine gate must be ****CONTROL/static-only****.

23

24 ---

25

26 **## 1. Problem & goal**

27

28 The rally-quality measurement is split across two machines by capability (``docs/GOLDEN_DATA_SHARING.md``):

29 - ``backend/eval/fusion_golden.py`` ****extracts**** ``*_golden_features.json`` ? needs the ****video + trajectory CSV + labels + GPU/WSL**** (GPU box only).

30 - ``backend/eval/fusion_compare.py`` / ``backend/eval/ablation.py`` ****re-score**** those JSONs on ****pure CPU, no video**** ? any machine.

31

32 The CPU re-scoring is **already** video-free, but the inputs live in gitignored ``output/`` / private OneDrive, so the regression tests that depend on them ****skip everywhere**** (``tests/test_ablation.py::test_litmus_reproduces_gate_a``, ``tests/test_fusion_compare.py``). The committed ``eval_baselines/`` baselines exist but ``regression.py`` still scores DB-stored predictions (needs a pipeline run on the video).

33

34 ****Goal:**** commit a tiny, version-tagged, ****numeric-only, non-attributable**** per-fixture corpus + auto-discovered gates so a clean ``git clone`` / CI gets a real red/green regression signal with ****zero video/GPU/DB****, while never leaking attributable or sensitive data.

35

36 ---

37

38 **## 2. Decisions locked**

39

#	Decision	Source	Implication
D1	**Corpus is owner-recorded / owned**	owner answer 2026-06-16	Legal analysis takes **Fork A** (clean): no third-party copyright/ToS infringement; concern is trade-secret/moat + privacy. Strongest non-attributability (no public reference to correlate against).
D2	**Repo private now, may open-source later**	owner answer 2026-06-16	Design for **public-grade hygiene from day one** ? git history is permanent; nothing scrubable-later.
D3	**Adults, consent not formalized**	owner answer 2026-06-16	**Person/pose/gait streams stay OUT of the public tier** (DPDP/GDPR derived-personal-data risk without consent). Public tier = **shuttle + abstract intervals + non-biometric features only** .
D4	**Freeze seam = post-detector trajectory**	<i>*(design)*</i>	workflow-1 synthesis Widest deterministic surface; reuse the shipped eval stack verbatim; the raw per-frame CSV is **never committed** (only ~5 aggregate, fps/res-invariant shuttle scalars/window).
D5	**Cross-machine gate is CONTROL/static-only**	<i>*(design)*</i>	both red-teams [verified cause] Learned variants (numpy/BLAS) are **owner-box/Tier-1 only** , asserted with tolerance, never as a cross-machine byte-snapshot.
D6	**Non-attributability = deletion-at-source, encryption = public-only barrier**	<i>*(design)*</i>	workflow-2 synthesis A contributor who runs the tests necessarily reads

plaintext+key (DRM/analog-hole). So we **delete** attribution, not hide it. Encryption only stops the no-key public. |

48 | D7 | **Two tiers** ***(design)*** | workflow-2 synthesis | **Tier-0** public/no-key (shuttle-only, de-attributed). **Tier-1** key-gated/out-of-repo (person/audio, full corpus), **skips** cleanly when absent*****. |

49

50 ---

51

52 ## 3. Legal foundation (researched ? gates on counsel, §8)

53

54 **Core verdict** (settled across IN/US/EU): **the derived numbers ? per-frame shuttle (x,y), rally start/end intervals, audio/court/aggregate-person feature values ? do NOT inherit the source video's copyright.** They are uncopyrightable ***facts/measurements***, not authored expression, and not a §101 "derivative work."

55

56 | Jurisdiction | Copyright of the data | Database right | Contract / ToS | Net |

57 |---|---|---|---|

58 | **India** (primary) | Not copyrightable as facts (**EBC v. D.B. Modak**); but compilations get thin protection on a low "skill/labour" bar (**Burlington**, **OLX v. Padawan**) ? only bites if **lifted from a 3rd-party dataset**, not self-derived | **None** (no sui-generis right) ? a real permissiveness edge | Anti-scraping ToS (untested in court) + **IT Act s.43/s.66** civil/criminal exposure for 3rd-party ingestion | **Owned-source + minors-excluded + no person/pose = clean** |

59 | **US** | Not copyrightable (**Feist**; §102(b); **NBA v. Motorola**) | None (**Feist** killed sweat-of-the-brow) | **Dominant risk.** ToS enforceable where copyright/CFAA fail (**ProCD**, **hiQ** ? injunction + **forced deletion**). Remedy for redistribution is takedown, not nominal damages | Publishable as facts **iff** owned/permissioned source + no barring ToS + no person/biometric/minor stream |

60 | **EU / UK** | Not copyrightable (**Football Dataco v. Yahoo**) | **Genuinely contestable.** **Football Dataco v. Sportradar**: recording pre-existing on-court facts = **"obtaining"**, not "creating" ? our "we created it" premise is **overconfident**; harm test = **CV-Online Latvia** | **Ryanair v. PR Aviation**: ToS can ban extraction/redistribution even with no IP right | Get EU+UK advice before redistributing numeric streams at scale |

61

62 **Provenance is the decisive fork.** **Fork A** (owner-recorded, unpublished) ? the only path safe to commit publicly; collapses to a trade-secret **choice** (Indian breach-of-confidence / Contract Act 1872 / IT Act s.72 ? **not** US DTSA) + privacy. **Fork B** (broadcast/YouTube/scraped) ? do **not** commit; the extraction act needs a license or now-**contested** US fair use (**Thomson Reuters v. Ross** 2025; USCO May-2025), and unlawful acquisition independently sinks it (**Bartz v. Anthropic**, ~\$1.5B settlement). Caveat: owner footage **shot at a third party's event** (tournament/club) carries ticket/venue/organisier/broadcaster terms ? treat as Fork B until cleared.

63

64 **Publish-by-stream (privacy):**

65 - ? **Publish-OK:** shuttle (x,y)+Visibility (inanimate object), abstract rally/GT **intervals** (identity-stripped), non-biometric audio/court/aggregate-person features detached from identity.

66 - ?? **Conditional** (effectively never in v1): **person bounding boxes that persistently single out a player** ? flips to GDPR Art.9 the moment it identifies. This architecture is prone to that flip ? treat per-player tracking as a tripwire.

67 - ? **Never publish:** pose/skeleton/**gait** (re-identifies at 80?95% even with faces blurred ? **Weiss et al. 2025**), face crops, any named-player?movement mapping. Matches the existing "gait/biometrics strictly future (GDPR Art.9)" guardrail.

68 - ? **Minors flip everything.** DPDP "child" = under 18; s.9(3) **prohibits behavioural monitoring** (the Rule-12 exemptions don't cover this project). A persistent per-player signature on a minor is, conservatively, prohibited. **Exclude minors** from any persisted stream beyond inanimate shuttle/interval data.

69

70 **Non-attributability is relative, not absolute.** A per-video trajectory is a fingerprint (de Montjoye: 4 points ? 95% re-ID). It's only non-attributable because the **Fork-A source stays unpublished** (no public reference to correlate against). Per **EDPS v. SRB** (CJEU C-413/23 P, 2025) it **remains personal data to the holder** who can re-run the detector. So: conditional, revocable (publish the source and the link snaps back), forward-looking (re-assess as the public-video environment grows). Never market it as "anonymous."

71

```

72      *Key authorities:* Feist 499 U.S. 340; NBA v. Motorola 105 F.3d 841; 17 U.S.C.
§101/§102(b); Sega v. Accolade 977 F.2d 1510; Football Dataco v. Yahoo (C-604/10) & v.
Sportradar (C-173/11); BHB v. William Hill (C-203/02); CV-Online Latvia (C-762/19); Ryanair v.
PR Aviation (C-30/14); Thomson Reuters v. Ross (D. Del. 2025); EBC v. D.B. Modak; DPDP Act 2023
s.3/s.9 + Rules 2025; EDPS v. SRB (C-413/23 P). Full URLs in the research artifact (§10).
73
74      ---
75
76      ## 4. Architecture (design)
77
78      ### 4a. Freeze seam + reuse map
79      Freeze at the trajectory CSV ? pure-Python replay. One shared helper
`backend/eval/golden_fixtures.py::replay_fixture()` is the single code path used by both the
freeze tool and both gates:
80      `parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates
? rally_gate net_crossings ? experiment.predict(CONTROL) ? merge_overlaps ? metrics.score`.
81      Reuses verbatim: `metrics.score/interval_iou`, `segmentation_metrics`,
`experiment.predict/compare/Variant.spec_hash`, `windowing.WindowingPreset` (default SERVED),
`regression.gt_hash/save_baseline + incommensurability guards`, `eval_stats` (seed=0),
`eval_baselines/` committed-baseline precedent.
82
83      ### 4b. Tiered model (SEALED-FIXTURES)
84      | Tier | Where | Key? | Contents | Test behaviour |
85      |---|---|---|---|---|
86      | Tier-0 | `eval_baselines/fixtures/` (tracked) | none | opaque `fixture_id`,
fps/frame_width (quantized), relative `start`/`end`, 5 shuttle scalars, label, relative
`gts`. No person/audio/pose. | Unconditional ? runs on every PR/fork; turns today's
skipped litmus into a real public gate |
87      | Tier-1 | OneDrive (default) or SOPS-encrypted in-repo | token/age key | full
corpus with person/audio; authoritative ?F1/CI/sign-test + exact Gate-A litmus | Skips
cleanly when key/data absent (generalize `golden_present()` ? `_tier1_present()`) |
88
89      A fresh no-key clone is always green; coverage never depends on a secret.
90
91      ### 4c. Anti-attribution measures (Tier-0)
92      - Opaque RANDOM surrogate id (`os.urandom` + a private surrogate?source map kept
off-git) ? not the MD5 `video_id`, and not HMAC(salt, video_id?name) (red-team: brute-
forceable over a tiny known roster if the salt leaks). Strip the `name` field (today = source
filename).
93      - Metadata strip ? no filename, video_id, match/event/player identity, capture date.
94      - Time-origin reset ? subtract per-fixture `t0` from all start/end/gts. Provably
metric-invariant (`metrics.interval_iou` and start/end-error are pure differences) ? bit-
identical scores, severed absolute timeline.
95      - Stream minimization ? Tier-0 keeps shuttle block only; person/audio excluded;
pose/gait/face have no field in the schema by design.
96      - Raw-trajectory exclusion ? never commit the per-frame `(Frame,Visibility,X,Y)`
CSV.
97      - Quantize fps/frame_width ? red-team: at n=6 the raw values are a camera/venue
quasi-identifier.
98      - Provenance hard-fail gate ? the de-attribution pass refuses to emit a public
fixture for any record not tagged owner-recorded(Fork-A) + minors-excluded + biometric-excluded.
99
100     ### 4d. The two gates
101     - Characterization/snapshot ? replay fixture, parse-compare JSON (never byte-
compare ? CRLF/float-format safe; `core.autocrlf=true` here), exact on int/structural fields,
`approx(abs=1e-6)` on floats; stamp + check windowing-preset/label/gt fingerprints
(incommensurable ? loud fail). Catches any behaviour change even at constant F1. CONTROL path
only across machines.
102     - Quality-floor ? score vs committed labels; assert per-video + mean F1 ? floor in
`eval_baselines/regression_baseline.json` (created here; carries a windowing fingerprint so
a served-default change is flagged incommensurable, not silently compared); emit `eval_stats`
paired-delta-CI + sign test (NUMBERS INFORM); add a metrics-vs-base-branch paired delta.
Synthetic-tier floors measure correctness, not field quality (documented).
103
104     ### 4e. Serialization & optional protobuf

```

```

105     JSON with `sort_keys=True` + pre-quantized floats + LF is sufficient. **Protobuf is
optional (01)** ? adopt only if cross-OS byte-stable snapshots are wanted (proto3
`rally.fixture.v1`, codegen + `protoc && git diff --exit-code` guard, `*.pb binary -text`).
Protobuf is **encoding, not secrecy** (trivially `--decode_raw`).
106
107     ### 4f. Encryption & keys (Tier-1 only)
108     **Default = out-of-repo + token** (existing OneDrive `RALLY_GOLDEN_DIR` seam ? zero
sensitive bytes in git history, sidesteps the legal "redistribution" act). **Alternative = SOPS
+ age** (per-recipient, revocable, value-level diffs) for Fork-A-clean-but-private data wanting
in-repo versioning. Rejected: git-crypt (whole-repo re-key), git-lfs (storage ? secrecy). CI
secret exposed **only to the Tier-1 job on trusted branches, never to fork-PR runners**. The
**HMAC/surrogate salt** (if used) and the surrogate?source map live with the private corpus,
rotated per release. Document explicitly that Tier-1 crypto defends **only** the no-key public.
109
110     ---
111
112     ## 5. Threat model (who's protected vs not)
113
114     | Actor | Protected against | NOT protected (by design / residual) |
115     |---|---|---|
116     | **TM1 no-key public** | map fixture?source/match/player; decrypt Tier-1; confirm a
source even holding the candidate file | read Tier-0 numbers + run shuttle-only harness
(intended; useless-in-isolation) |
117     | **TM2 keyed contributor** | recover filename/video_id/match/face/pose/gait ? *never
written* (survives the analog hole) | read every value the suite reads (unpreventable; not
pretended otherwise) |
118     | **TM3 CI runner** | leaked log can't leak attribution it never had | runs the suite
(intended); Tier-1 secret never on fork runners |
119     | **TM4 malicious keyed insider** | correlation needs a public reference; Fork-A source
is unpublished ? "not reasonably likely" | **residual:** conditional + revocable ? publish/leak
the source later and the link can snap back (forward-looking re-test duty) |
120     | **TM5 re-ID adversary** | source unpublished + only ~5 invariant scalars/window +
reset timeline ? no back-match | becomes live if a Fork-A source is ever published |
121     | **TM6 accidental plaintext commit** | required CI leak-scan rejects MD5/known-
name/non-surrogate/plaintext-SOPS | if the required check is disabled, a leak lands |
122
123     ---
124
125     ## 6. Honest limits ? what a green suite does NOT prove
126
127     - Green proves only that the **deterministic post-trajectory transforms** are unchanged
for the **frozen cases**. It does **not** cover: the WASB/TrackNet detector,
`_probe_fps_width`/cv2 frame-peek, the AI/provider handoff, chunking/re-encode, the ffmpeg
stitch, DB persistence/migrations, or the API filter. *A refactor that breaks decoding or the
detector passes these gates green.* The owner-box `regression.py` DB run remains the only full-
chain check.
128     - Characterization = **behaviour-locking, not correctness** ? it freezes current
behaviour *including current bugs*. "Passing" = "behaviour unchanged", never "behaviour
correct". This caveat must ride in the **test pass/fail message**, not only the docs.
129     - **Synthetic-tier floors = plumbing correctness, NOT field quality.** Never cite
synthetic F1 in `QUALITY_ITERATIONS.md` / the paper as rally-detection performance (tag `kind:
synthetic`).
130     - Tier-0 public coverage is **shuttle-only** ; the person/audio fusion ?F1 and exact
Gate-A values need Tier-1 (skipped on no-key clones).
131     - Data is **not "anonymous"** absolutely (EDPS v. SRB) ? relative, conditional,
revocable, forward-looking.
132     - Encryption gives **no secrecy from a running contributor** ; protobuf gives **no
secrecy at all**.
133     - De-attribution does **not** cure provenance/licensing ? it sits *behind* the per-
record provenance gate + counsel sign-off.
134     - `--refresh-snapshot` can rubber-stamp a real regression; mitigation is **review of the
visible diff** + a separate, deliberate floor-baseline update. Require a recorded reason.
135
136     ---
137

```

```

138    ## 7. Deliverables & progress tracker ? **source of truth**
139
140    Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. One **small PR per row**,
141    branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
142
143    | ID | Deliverable | Depends on | Counsel-gated? | Status | PR |
144    |----|-----|-----|-----|-----|----|
145    | **P0** | **Leak remediation + required CI leak-scan.** Scrub real names/paths in
146    `eval_baselines/heuristic_lovo_n6.json`, `gemini_wrapper_2026-06-10.json`,
147    `backend/eval/eval_partial_labels.py`, `tests/test_cleanup_caches.py` ? opaque surrogate keys +
148    private map; add CI leak-scan (MD5 / known-name / non-surrogate / plaintext-SOPS) as a
149    **required status check** ; decide on git history (PR #77, accept vs `filter-repo`). | ? | No
150    (owned data hygiene) | ? | ? |
151
152    | **M1** | **Fixture framework + synthetic Tier-0 + shared replay helper** (also
153    delivered M2-core + M3 ? see below). `backend/eval/golden_fixtures.py` (`replay_fixture`, schema
154    dispatch, `Path(__file__)`-relative) + 3 **tool-generated** synthetic fixtures + manifest +
155    README + `.gitattributes` LF pin + `tests/test_golden_fixtures.py`. CONTROL-only / pure-stdlib;
156    cross-OS verified. | ? | No (synthetic) | ? | [#189](https://github.com/avidullu/badminton-
157    highlight-indexer/pull/189) |
158
159    | **M2** | **Freeze/refresh extractor.** Core (`freeze_synthetic` / `refresh_snapshot` /
160    `--check` CLI + idempotence test) shipped in M1 (#189); the `--license`/`--minors-
161    free`/`--owned` **refusal gate** shipped with P1 (this PR). | M1 | No | ? |
162    [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) · _this PR_ |
163
164    | **M3** | **Characterization gate** ? `tests/test_golden_fixtures.py`: parse-compared
165    (exact-int / approx-float 1e-6), **CONTROL-only** , positive no-person/audio assertion,
166    perturbation + idempotence. **Shipped in M1 (#189).** (Windowing/label fingerprint guard + row-
167    count cap ? fold into M4.) | M1 | No | ? | [#189](https://github.com/avidullu/badminton-
168    highlight-indexer/pull/189) |
169
170    | **MX** | **Cross-OS determinism guard (CI).** Lightweight `golden-crossos` CI job
171    (ubuntu/windows/macOS) running `--check` + the fixtures test ? locks the cross-OS guarantee
172    (CONTROL replay is numpy-only, no GPU stack; a *separate fast job*, not full-suite×3). | M1 | No
173    | ? | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) |
174
175    | **M4** | **Quality-floor gate + first `regression_baseline.json`.**
176    `tests/test_golden_quality_floor.py`; create `eval_baselines/regression_baseline.json` with
177    **windowing fingerprint** ; metrics-vs-base paired-delta; corpus-level (not per-slug) LOVO for
178    runtime. | M1, M2 | No | ? | ? |
179
180    | **P1** | **De-attribution + minimization** ? `golden_fixtures.freeze_real_fixture()`
181    (extends the M1 module, not a separate script): **opaque random `fx<hex>` surrogate slug** ,
182    metadata strip, metric-invariant **time-origin reset** , strict GT loader, **positive no-
183    person/audio/MD5/source-path assertion** , manifest non-clobber; `--freeze-real` CLI with the
184    **refusal gate** (exit 2 unless owned + minors-free + license). Tooling-only, 12 synthetic
185    tests; **no real data committed** . | M1 | No (tooling) | ? | _this PR_ |
186
187    | **P2** | **Owner-accepted (researched-risk), 2026-06-16.** Owner accepted the
188    researched IP/privacy analysis (§3) to proceed **under its conditions** (owned/Fork-A source ·
189    minors excluded · biometric/person streams excluded · de-attributed). Owner's informed risk
190    acceptance ? **NOT a retained-counsel opinion** ; optional future counsel confirmation per §8. |
191    P1 | Owner-accepted | ? | ? |
192
193    | **P3** | **Tier-0 REAL subset.** Commit de-attributed, minors/biometric-excluded,
194    owned Fork-A real fixtures (unblocked by P2). Hard-gated **in code** by P1's refusal gate (owned
195    + minors-free + license required) ? needs the owner's per-video clearance list. | P1, P2 |
196    conditions §3 | ? | ? |
197
198    | **O1** | *(optional)* **Protobuf + deterministic serialization.** `rally.fixture.v1`
199    proto3, codegen + `protoc`/`git diff --exit-code` guard, canonical writer, `.gitattributes`
200    binary. Only if cross-OS byte-stable snapshots are wanted. | M1 | No | ? | ? |
201
202    | **O2** | *(optional)* **Tier-1 key-gated path.** `scripts/fetch_golden.py` (OneDrive
203    token, default) and/or SOPS+age; `_tier1_present()` ; `RALLY_GOLDEN_DIR` wired; Tier-1 CI job
204    gated on secret, never on fork runners. | M1 | No | ? | ? |
205
206    | **DOC** | **Finalize this doc + cross-links.** Add `eval_baselines/fixtures/` row to
207    `CODE_MAP §0` + TEST MAP; cross-link `EVALUATION.md` / `EXPERIMENT_HARNESS.md` /
208    `GOLDEN_DATA_SHARING.md` / `QUALITY_ITERATIONS.md` ; update `docs/README.md` index + memory
209    `current-state`/handoff. | M1..M4 | No | ? | ? |
210
211
212    **Must-fix-before-any-public-commit (red-team), folded into the rows above:** ? numeric-
213    tolerance not byte-equality + multi-OS matrix + CONTROL-only (M3/D5/MX ?) ; ? scrub names in
214    **all** files + history decision (P0) ; ? leak-scan a **required** check, hooks are convenience

```

only (P0); ? person-optional loader + **positive** no-person assertion, don't trust the silent
`0.0` default (P1 ?); ? random surrogate id not HMAC-over-roster (P1 ?); ? quantize/drop
fps/frame_width (P1 ? fps/fw retained as scale constants; quantize is a P3 option); ? document
Tier-0 omits the fusion path (DOC); ? counsel + Fork-A + minors/biometric exclusion as hard
preconditions (P2 owner-accepted; enforced in code by P1's refusal gate ?).

158

159 **### Progress log** (quantitative ? coverage must hold/improve, CI incl. the 3-OS cross-OS
job green)

160 | PR | Deliverable(s) | Suite | Coverage | Notes |

161 |----|-----|-----|-----|-----|

162 | [#189](https://github.com/avidullu/badminton-highlight-indexer/pull/189) | M1 +
M2-core + M3 | 746 pass / 7 skip | 80.41% | synthetic Tier-0 + CONTROL replay +
characterization; cross-OS proven (CI Linux replayed Windows-frozen fixtures) |

163 | [#191](https://github.com/avidullu/badminton-highlight-indexer/pull/191) | MX +
tracker/legal reconcile | full suite green | 80.41% | `golden-crossos` CI job green on
ubuntu+windows+macos ? cross-OS **enforced**; legal ? owner-accepted |

164 | _this PR_ | P1 + M2 refusal gate | 759 pass / 7 skip | **80.49%** (+0.08) |
`freeze\_real\_fixture` + refusal gate; **+13 tests**; tooling-only, no real data committed |

165

166 ---

167

168 **## 8. Open owner / counsel questions**

169

170 **Counsel questions ? documented basis** (the owner accepted the researched risk on
2026-06-16; see P2). These remain the record, and a retained-counsel confirmation is
optional/recommended before scaling beyond a small owned, minors-free, de-attributed subset:

171 1. Does the EU/UK **sui-generis** database right¹ subsist in our self-derived
shuttle/rally streams on the *Sportradar* "obtained vs created" reasoning, and does
redistributing a de-attributed Tier-0 subset cause *CV-Online* harm?

172 2. Post *Thomson Reuters v. Ross* + USCO-2025, is **US** fair use for the extraction
step² defensible if the dataset could compete in a "data/analytics" market ? does publishing
only numbers (never footage) matter?

173 3. India: are anti-scraping ToS enforceable; **IT Act s.43/s.66** exposure for any 3rd-
party ingestion; does pending *ANI v. OpenAI* (~Apr 2026) move it? Confirm **DPDP s.9/Rule 10**³
and whether per-player tracking is prohibited s.9(3) monitoring of a minor.

174 4. Sign-off on the **final committed Tier-0 subset + provenance log**; is non-
attributability-via-private-source a defensible posture to assert given its
conditional/revocable nature?

175

176 **Owner (strategic):**

177 5. Publishing the Fork-A subset **forfeits the trade-secret moat** (Indian breach-of-
confidence framing) ? weighed against community/benchmark/paper aims. Your call.

178 6. **Forward-looking commitment**: do not publish a highlight reel of the *same* source
whose fixtures are committed (without re-clearing), and accept a periodic re-assessment as the
public-video environment grows.

179 7. **Scope of first public commit**: synthetic-only (zero provenance risk, recommended)
vs a counsel-cleared real subset.

180 8. **Tier-1 mechanism**: out-of-repo+token (recommended) vs SOPS+age in-repo. And: is
protobuf (01) wanted, or is canonical-JSON enough?

181

182 ---

183

184 **## 9. Definition of done**

185

186 The project is **DONE** (flip `Status ? DONE`, archive) when:

187 - [] **P0, M1?M4, P1, DOC** are ? and merged.

188 - [] A clean `git clone` on **win + ubuntu + mac** runs `python run\_all\_tests.py` and
the **Tier-0** gates pass green with no key / no video / no GPU / no DB.

189 - [] The **CI leak-scan** is a required status check⁴ and passes (no MD5 / real-name /
non-surrogate / plaintext-SOPS in the tree).

190 - [] Characterization + quality-floor gates are **parse-compared, CONTROL-only,**
windowing-fingerprinted⁵, and demonstrably catch a seeded behaviour change.

191 - [] `docs/CODE\_MAP.md` §0 + TEST MAP updated; this doc cross-linked; `NEXT\_STEPS.md` +
memory `current-state` point here.

192 - [] **Either** the public commit is synthetic-only, **or** (**P2 owner-accepted** ? +

```

P3**) for a real-data commit, under the S3 conditions enforced **in code** by P1's refusal gate.
193   - [ ] S6 honest-limits caveat rides in the test pass/fail messages, not only the docs.
194
195   *Optional 01/02 may remain ? without blocking DONE (they're earned, not assumed).*
196
197   ---
198
199   ## 10. References
200
201   **Internal code anchors (verified):** `backend/eval/{regression.py,fusion_golden.py,fusi
on_compare.py,ablation.py,experiment.py,metrics.py>windowing.py,distill_local.py,gt_loader.py,ev
al_stats.py}` · `backend/pipeline/detectors/{base.py,tracknet_runner.py,rally_gate.py}` ·
`backend/utils/hashing.py::compute_video_id` · `eval_baselines/{heuristic_lovo_n6.json,gemini_wr
apper_2026-06-10.json,experiment_leaderboard.json}` ·
`tests/test_ablation.py::test_litmus_reproduces_gate_a` · `.github/workflows/ci.yml`.
202   **Internal docs:** `GOLDEN_DATA_SHARING.md` (#175), `EVALUATION.md`,
`EXPERIMENT_HARNESS.md`, `QUALITY_ITERATIONS.md`, `MULTI_SIGNAL_FUSION_PLAN.md`, `CODE_MAP.md
S0`.
203   **Legal authorities:** see S3 (full URLs in the research workflow artifact). *This
section is researched general information, not legal advice.*
204
205   ---
206
207   ### Changelog
208   - **2026-06-16** ? Initial draft synthesized from two multi-agent workflows (mechanics
design + 2 critiques; legal memo IN/US/EU + SEALED-FIXTURES architecture + red-team). Verified
the live name-leak and the BLAS-determinism cause against code. Status: DRAFT, owner-gated.
209

```

23. assistant (2026-06-16T06:27:40.530Z)

Now let me look at the committed fixtures, the manifest, the README, and the .gitattributes for LF pinning. Let me also check the gemini_refine merge_overlaps and classifier to understand the CONTROL path's numpy avoidance claim.

24. user (2026-06-16T06:27:42.722Z)

```

total 28
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 ..
-rw-r--r-- 1 avidu 197609 4018 Jun 16 11:54 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 clean_single_rally
-rw-r--r-- 1 avidu 197609 1557 Jun 16 11:54 manifest.json
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 multi_rally_with_deadtime
drwxr-xr-x 1 avidu 197609    0 Jun 16 11:54 occlusion_break
---MANIFEST---
[
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "clean_single_rally",
      "expected": "clean_single_rally/expected.json",
      "labels": "clean_single_rally/labels.csv",
      "provenance": "clean_single_rally/provenance.json",
      "trajectory": "clean_single_rally/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "clean_single_rally",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  }
]

```

```

    }
  },
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "multi_rally_with_deadtime",
      "expected": "multi_rally_with_deadtime/expected.json",
      "labels": "multi_rally_with_deadtime/labels.csv",
      "provenance": "multi_rally_with_deadtime/provenance.json",
      "trajectory": "multi_rally_with_deadtime/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "multi_rally_with_deadtime",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  },
  {
    "fps": 30.0,
    "frame_width": 1280.0,
    "kind": "synthetic",
    "paths": {
      "dir": "occlusion_break",
      "expected": "occlusion_break/expected.json",
      "labels": "occlusion_break/labels.csv",
      "provenance": "occlusion_break/provenance.json",
      "trajectory": "occlusion_break/trajectory.csv"
    },
    "schema_version": 1,
    "slug": "occlusion_break",
    "windowing": {
      "max_win": 0.0,
      "mg": 2.0,
      "mwd": 2.0,
      "vt": 0.02
    }
  }
]

```

---SLUG DIRS---

eval_baselines/fixtures/clean_single_rally/:

expected.json
 labels.csv
 provenance.json
 trajectory.csv

eval_baselines/fixtures/multi_rally_with_deadtime/:

expected.json
 labels.csv
 provenance.json
 trajectory.csv

eval_baselines/fixtures/occlusion_break/:

expected.json
 labels.csv
 provenance.json
 trajectory.csv

25. user (2026-06-16T06:27:43.111Z)

1 ""Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).

```

2
3 WASB (shuttle-only) over-fires on between-point service feeds. This drives the
4 EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
5 each candidate window is cut to a small padded clip and handed to Gemini, which ?
6 seeing the actual footage ? returns the true rally inside it (tight boundaries) or
7 nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
8 not GB-scale chunks; high-precision because Gemini judges each candidate visually.
9
10 Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
11 sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,
12 calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.
13
14 This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
15 trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.
16
17 Run (GEMINI_API_KEY in env):
18     python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv
19 \
20     --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
21
22 from __future__ import annotations
23
24 import os
25 import os.path as osp
26 import time
27 import argparse
28 import concurrent.futures
29 from typing import List, Tuple, Optional
30
31 from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
32 from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
33 from backend.utils.video import get_video_duration, extract_video_chunk
34 from backend.sports import sport_registry
35 from backend.providers import provider_registry
36
37 Interval = Tuple[float, float]
38
39 def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
40     """Union of overlapping intervals, optionally stitching across a small gap.
41
42     gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
43     shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
44     gaps
45     (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
46     logic,
47     which over-merged multiple real rallies WASB had bundled into one candidate.)"""
48     merged: List[Interval] = []
49     for s, e in sorted(intervals):
50         if merged and s <= merged[-1][1] + gap_tol:
51             merged[-1] = (merged[-1][0], max(merged[-1][1], e))
52         else:
53             merged.append((s, e))
54     return merged
55
56 def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
57     """Map a Gemini clip's returned segments back to full-video seconds, clamped to
58     clip."""
59     out: List[Interval] = []
60     for sg in raw or []:
61         try:
62             st = float(sg["start_time"]) + clip_start
63             en = float(sg["end_time"]) + clip_start

```

```

62         except (TypeError, ValueError, KeyError):
63             continue
64         st = max(clip_start, st)
65         en = min(clip_end, en)
66         if en > st:
67             out.append((round(st, 3), round(en, 3)))
68     return out
69
70
71     def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,
72                     pad: float, duration: float, tmpdir: str, idx: int,
73                     clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval,
74 List[Interval]]:
75     # Own provider/client per call: the genai client isn't reliably thread-safe, and a
76     # shared one across workers throws socket errors. A fresh lazy client is cheap.
77     provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
78     s, e = window
79     cs = max(0.0, s - pad)
80     ce = min(duration, e + pad) if duration > 0 else e + pad
81     clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
82     # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
83     # 500MB upload cap even for 4K/5K sources.
84     if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True,
85 scale_width=clip_scale_width):
86         return idx, window, []
87     try:
88         prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
89         raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
90 label=f"cand{idx}")
91         mapped = _offset_segments(raw, cs, ce)
92     finally:
93         try:
94             os.remove(clip)
95         except OSError:
96             pass
97     # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
98     # (merge_gap bridged the between-point gaps), and collapsing them to one envelope
99     swept
100     # dead time into a "rally" (precision loss). Tiny invisibility splits are healed
101     later
102     # by the gap-tolerant global merge in run().
103     return idx, window, mapped
104
105
106     def run(video: str, trajectory: str, fps: float, frame_width: float,
107            cfg=BASELINE, pad: float = 3.0, model: str = "gemini-flash-latest",
108            max_workers: int = 4, out_csv: str = "output/gemini_refined.csv",
109            min_dur: float = 2.0, limit: int = 0, clip_scale_width: Optional[int] = 1280) ->
110 dict:
111     api_key = os.environ.get("GEMINI_API_KEY")
112     if not api_key:
113         raise SystemExit("GEMINI_API_KEY not set in env")
114     sport_profile = sport_registry.get("badminton")
115     duration = get_video_duration(video)
116
117     candidates = make_predict_fn(trajectory, fps, frame_width)(cfg)
118     if limit:
119         candidates = candidates[:limit]
120     tmpdir = osp.join("output", "gemini_refine_clips")
121     os.makedirs(tmpdir, exist_ok=True)
122
123     print(f"[gemini_refine] {len(candidates)} WASB candidates (cfg {cfg}) -> Gemini
124 {model}, "
125         f"pad {pad}s, {max_workers} workers", flush=True)
126     t0 = time.time()

```

```

120     per_candidate = []
121     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
122         futs = [ex.submit(refine_window, api_key, model, sport_profile, video, w, pad,
duration,
123                         tmpdir, i, clip_scale_width)
124                 for i, w in enumerate(candidates)]
125     for fut in concurrent.futures.as_completed(futs):
126         idx, window, mapped = fut.result()
127         per_candidate.append((idx, window, mapped))
128         tag = "REJECT (no rally)" if not mapped else \
129             " ".join(f"{seconds_to_mmss(a)}-{seconds_to_mmss(b)}" for a, b in
mapped)
130         print(f" cand {idx:>2}
{seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)
131
132     raw_intervals = [iv for _, _, m in per_candidate for iv in m]
133     refined = [(s, e) for s, e in merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >=
min_dur]
134     write_segments_csv(out_csv, refined)
135     try:
136         os.rmdir(tmpdir)
137     except OSError:
138         pass
139
140     kept_cands = sum(1 for _, _, m in per_candidate if m)
141     play = sum(e - s for s, e in refined)
142     print(f"\n[gemini_refine] {kept_cands}/{len(candidates)} candidates had a rally; "
143         f"{len(refined)} merged rallies, {play:.1f}s play -> {out_csv} "
144         f"({time.time() - t0:.0f}s)", flush=True)
145     return {"refined": refined, "num_candidates": len(candidates),
146           "candidates_with_rally": kept_cands, "out_csv": out_csv}
147
148
149     def full_video_rallies(video: str, model: str = "gemini-flash-latest",
150                           out_csv: str = "output/gemini_fullvideo.csv", clip_scale_width:
Optional[int] = 1280,
151                           min_dur: float = 2.0, chunk_sec: float = 120.0, overlap: float =
10.0,
152                           max_workers: int = 3) -> List[Interval]:
153         """Full-context pass: hand Gemini the WHOLE clip (downscaled, chunked to stay under
the
154         upload cap), so it finds every rally itself ? not bounded by WASB's candidate
recall.
155         Independent of the trajectory; complements refine()."""
156         api_key = os.environ.get("GEMINI_API_KEY")
157         if not api_key:
158             raise SystemExit("GEMINI_API_KEY not set in env")
159         sport_profile = sport_registry.get("badminton")
160         duration = get_video_duration(video)
161         starts, t = [], 0.0
162         while t < duration:
163             starts.append(t)
164             t += max(1.0, chunk_sec - overlap)
165         tmpdir = osp.join("output", "gemini_full_clips")
166         os.makedirs(tmpdir, exist_ok=True)
167
168         def do_chunk(ci: int, cstart: float) -> List[Interval]:
169             cend = min(cstart + chunk_sec, duration)
170             clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
171             if not extract_video_chunk(video, cstart, cend - cstart, clip, reencode=True,
scale_width=clip_scale_width):
172                 return []
173             try:
174                 provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)

```

```

175         prompt = sport_profile.get_prompt(chunk_duration=cend - cstart)
176         raw, _ = provider.analyze_video(clip, prompt, chunk_duration=cend - cstart,
label=f"chunk{ci}")
177         return _offset_segments(raw, cstart, cend)
178     finally:
179         try:
180             os.remove(clip)
181         except OSError:
182             pass
183
184     print(f"[gemini_full] {len(starts)} chunk(s) of {video} (<= {chunk_sec}s, width
{clip_scale_width}) "
185           f"-> Gemini {model}", flush=True)
186     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
187         chunks = [f.result() for f in [ex.submit(do_chunk, i, s) for i, s in
enumerate(starts)]]
188     rallies = [(s, e) for s, e in merge_overlaps([iv for c in chunks for iv in c],
gap_tol=1.0) if e - s >= min_dur]
189     write_segments_csv(out_csv, rallies)
190     try:
191         os.rmdir(tmpdir)
192     except OSError:
193         pass
194     play = sum(e - s for s, e in rallies)
195     print(f"[gemini_full] {len(rallies)} rallies, {play:.1f}s play -> {out_csv}",
flush=True)
196     return rallies
197
198
199     def main() -> None:
200         ap = argparse.ArgumentParser(description="Refine WASB candidates with Gemini.")
201         ap.add_argument("--video", required=True)
202         ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required
unless --full-video)")
203         ap.add_argument("--fps", type=float, default=59.94)
204         ap.add_argument("--frame-width", type=float, default=1920.0)
205         ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
206             help="WASB candidate config to feed Gemini (baseline = whole
rallies, good recall)")
207         ap.add_argument("--pad", type=float, default=3.0)
208         ap.add_argument("--model", default="gemini-flash-latest")
209         ap.add_argument("--max-workers", type=int, default=4)
210         ap.add_argument("--min-dur", type=float, default=2.0)
211         ap.add_argument("--out", default="output/gemini_refined.csv")
212         ap.add_argument("--limit", type=int, default=0, help="only the first N candidates
(smoke test)")
213         ap.add_argument("--clip-scale-width", type=int, default=1280,
214             help="downscale uploaded clips to this width (0=keep source res)")
215         ap.add_argument("--full-video", action="store_true",
216             help="full-context pass: send the whole clip (chunked/downscaled) to
Gemini "
217                 "instead of refining WASB candidates (needs no --trajectory)")
218         args = ap.parse_args()
219         scale = args.clip_scale_width or None
220         if args.full_video:
221             full_video_rallies(args.video, model=args.model, out_csv=args.out,
222                 clip_scale_width=scale, min_dur=args.min_dur,
max_workers=args.max_workers)
223         return
224     if not args.trajectory:
225         raise SystemExit("--trajectory is required unless --full-video")
226     cfg = BASELINE if args.config == "baseline" else TUNED
227     run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
228         model=args.model, max_workers=args.max_workers, out_csv=args.out,
229         min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)

```

```

230
231
232     if __name__ == "__main__":
233         main()
234

```

26. assistant (2026-06-16T06:27:45.870Z)

Now let me look at the committed expected.json files and the README, plus the .gitattributes for LF pinning and the classifier (to confirm CONTROL avoids numpy import).

27. user (2026-06-16T06:27:49.670Z)

```

=== clean_single_rally/expected.json ===

```

```

{
  "candidates": [
    {
      "end": 3.966667,
      "shuttle": {
        "active_ratio": 1.008475,
        "duration": 3.933333,
        "mean_velocity": 1.27678,
        "net_crossings": 7,
        "peak_velocity": 1.970041
      },
      "start": 0.033333
    }
  ],
  "control_segments": [
    [
      0.033333,
      3.966667
    ]
  ],
  "gt_hash": "6c93801fd607",
  "iou": 0.5,
  "schema_version": 1,
  "score": {
    "f1": 1.0,
    "false_negatives": 0,
    "false_positives": 0,
    "iou_threshold": 0.5,
    "mean_end_error": 0.033333,
    "mean_iou": 0.983333,
    "mean_start_error": 0.033333,
    "num_gt": 1,
    "num_pred": 1,
    "precision": 1.0,
    "recall": 1.0,
    "true_positives": 1
  },
  "slug": "clean_single_rally"
}

```

```

=== provenance ===

```

```

{
  "description": "One continuous ~4s rally; shuttle oscillates across the net ? 1 window.",
  "fps": 30.0,
  "frame_width": 1280.0,
  "iou": 0.5,
  "kind": "synthetic",
  "license": "synthetic",
  "minors_free": true,
  "note": "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. Plumbing-correctness fixture ? NOT a measure of rally-detection quality ($6).",
  "paths": {

```

```

    "expected": "expected.json",
    "labels": "labels.csv",
    "trajectory": "trajectory.csv"
  },
  "schema_version": 1,
  "slug": "clean_single_rally",
  "windowing": {
    "max_win": 0.0,
    "mg": 2.0,
    "mwd": 2.0,
    "vt": 0.02
  }
}
=== labels ===
start,end
0.000000,4.000000
=== README ===

```

`eval\_baselines/fixtures/` ? Golden Regression Fixtures (Tier-0, synthetic)

Video-free, **CONTROL-only** characterization fixtures for the rally-segmentation pipeline. A contributor on **any machine ? no video, GPU, WSL, or DB** ? replays these committed structured files through the already-shipped eval stack and gets a red/green regression signal. See ``docs/GOLDEN_REGRESSION_FIXTURES.md`` (deliverable **M1**; §4, §6, §7).

What's here

Each fixture is one directory:

```

| File | Purpose |
|-----|-----|
| `trajectory.csv` | Synthetic shuttle trajectory (`Frame,Visibility,X,Y`) ? the freeze seam input. |
| `labels.csv` | Synthetic ground-truth rally intervals (`start,end`). |
| `expected.json` | Frozen replay snapshot (candidates + 5 shuttle features + CONTROL segments + score). |
| `provenance.json` | Pinned windowing params + privacy/licensing tags + description. |

```

``manifest.json`` lists every fixture; ``backend/eval/golden_fixtures.py`` is the shared replay module + freeze/refresh/check CLI.

Guardrails (non-negotiable)

- **Synthetic / owned only.** Every fixture is generated by deterministic formulae (NO random ? zero third-party footage, zero provenance/licensing risk. Tagged ``kind: "synthetic"`, `license: "synthetic"`, `minors_free: true``.
- **No biometric / identity streams.** Tier-0 carries the **5 shuttle feature columns only** (``duration, peak_velocity, mean_velocity, active_ratio, net_crossings``). There is, by design, **no person / pose / gait / face / audio / player field anywhere in the schema** ? a test asserts this positively (deletion-at-source, §4c). Minors and biometrics are excluded.
- **CONTROL / pure-stdlib only.** The replay uses ``experiment.CONTROL`` (``is_learned() == False``) ? no numpy / logistic-regression. Learned variants' near-threshold decisions flip across CPUs, so they can **never** anchor a cross-machine snapshot (§4d / D5).
- **Parse-compare, never byte-compare.** The gate ``json.load``'s both sides: exact equality on int/structural fields, abs-tolerance ``1e-6`` on floats (CRLF/float-format safe).
- **Windowing is pinned per fixture.** The ``(vt, mg, mwd, max_win)`` params live in each ``provenance.json`` and are passed explicitly to ``build_candidates`` ? the replay does **not** depend on the named ``windowing.SERVED`` constant, so a future SERVED retune can't silently churn fixtures.

Fixtures are GENERATED, never hand-edited

Do **not** hand-edit ``expected.json`` / ``provenance.json`` / the CSVs. They are produced correct-by-construction by the CLI.

```
```bash
```

**Generate the synthetic trajectory.csv + labels.csv for all built-in scenarios, then freeze expected.json + provenance.json + (re)write manifest.json:**

```
python -m backend.eval.golden_fixtures --freeze-synthetic
```

**Replay all fixtures and parse-compare vs the committed expected.json (exit 0/1):**

```
python -m backend.eval.golden_fixtures --check
```

**Re-freeze expected.json from the COMMITTED trajectory+labels (NO regeneration). ? rubber-stamp a real regression ? review the visible diff and record a reason (§6):**

```
python -m backend.eval.golden_fixtures --refresh-snapshot --all --reason "<why>"
```

```
python -m backend.eval.golden_fixtures --refresh-snapshot --slug clean_single_rally --reason
```

```
"<why>"
```

```
```
```

Honest limits (§6 ? rides in the test pass/fail messages, not only here)

- A green suite proves only that the ****deterministic post-trajectory transforms**** are unchanged for the frozen cases. It does ****not**** cover the WASB/TrackNet detector, frame decoding, the AI/provider handoff, chunking, the ffmpeg stitch, DB persistence, or the API filter.
- Characterization is ****behaviour-locking, not correctness**** ? it freezes current behaviour ***including current bugs***. "Passing" = "behaviour unchanged", never "behaviour correct".
- ****Synthetic-tier F1 measures plumbing correctness, NOT field quality.**** Never cite a synthetic fixture's F1 in `QUALITY\_ITERATIONS.md` or the paper as rally-detection performance.

28. user (2026-06-16T06:27:51.009Z)

```
=== .gitattributes ===
```

Golden Regression Fixtures (M1) ? pin LF line endings so the committed fixtures are byte-stable across OSES despite core.autocrlf. The characterization gate parse-compare (never byte-compares) JSON, but a stable on-disk form keeps `git diff` reviewable and fixtures reproducible. See docs/GOLDEN_REGRESSION_FIXTURES.md §4d/§4e.

```
eval_baselines/fixtures/** text eol=lf
```

```
=== fixtures .gitattributes ===
```

```
=== classifier head ===
```

```
"""A tiny, dependency-free logistic-regression classifier (numpy only).
```

The offline learned segmenter (ADR-004) must run locally with no external API and no heavy ML stack ? the project already depends on numpy, and sklearn is not installed. This implements just enough: standardized features, balanced-class gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained model is a small, human-readable, version-controllable file.

It deliberately stays simple: the rally-window features are few and roughly linearly separable (high player motion + sustained activity => real rally), so a well-regularized linear model is a sensible, inspectable baseline. Swap in a richer model later behind the same fit/predict_proba/save/load interface.

```
"""
```

```
import json
```

```
from typing import List, Optional
```

```
import numpy as np
```

```
class LogisticRegression:
```

```
    def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
```

```
        self.lr = lr
```

```

self.epochs = epochs
self.l2 = l2
self.w: Optional[np.ndarray] = None
self.b: float = 0.0
self.mean: Optional[np.ndarray] = None
self.std: Optional[np.ndarray] = None
self.feature_names: Optional[List[str]] = None

@staticmethod
def _sigmoid(z: np.ndarray) -> np.ndarray:
    # Clip for numerical stability (avoid overflow in exp).
    return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

def _standardize(self, X: np.ndarray) -> np.ndarray:
    return (X - self.mean) / self.std

def fit(self, X, y, feature_names: Optional[List[str]] = None,

```

29. assistant (2026-06-16T06:27:58.149Z)

Now I have a comprehensive picture. Let me verify a few critical things: (1) whether `experiment.py` imports numpy at module level (the CONTROL "no numpy execution" claim ? but the import itself happens), (2) the key robustness concern about the `max\_win` parameter not being passed through the replay, and (3) what happens with the net-crossings gate (`min\_crossings=0`) interaction.

Let me check the critical robustness gap: the replay calls `build\_candidates(proposal=proposal)` where `proposal` is a 3-tuple `(vt, mg, mwd)`. The `max\_win` (PINNED_MAX_WINDOW) is pinned in provenance but **never passed** to `build\_candidates`, because `build\_candidates` doesn't accept `max\_window\_duration`.

30. user (2026-06-16T06:27:59.698Z)

No matches found

31. user (2026-06-16T06:28:00.757Z)

No matches found

32. assistant (2026-06-16T06:28:05.813Z)

Confirmed: `distill\_local.build\_candidates` does NOT accept or pass `max\_window\_duration`. This is the core robustness gap I need to analyze in detail. Let me verify the exact signature and what happens if someone enables bounded-windowing (W1, PR #187) via SERVED.

Let me trace the precise control flow for the windowing-change scenarios the lens asks about. Let me also confirm whether `predict(CONTROL, vc)` ignores `net\_crossings` entirely (since `min\_crossings=0` for CONTROL).

33. user (2026-06-16T06:28:06.783Z)

```

52     def build_candidates(trajjectory_csv: str, fps: float, frame_width: float,
53                         proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
54         """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
55         points = TrackNetRunner.parse_trajectory_csv(trajjectory_csv)
56         vt, mg, mwd = proposal
57         wins = TrackNetRunner.trajectory_to_action_windows(
58             points, fps=fps, frame_width=frame_width,
59             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
60         intervals = [(w["start"], w["end"]) for w in wins]
61         gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings

```

```

62     X: List[List[float]] = []
63     for w, g in zip(wins, gated):
64         dur = max(1e-6, w["end"] - w["start"])
65         win_frames = max(1.0, dur * fps)
66         X.append([
67             dur,                                     # rally length (sec) ?
68             float(w["peak_velocity"]),               # already normalized by
69             float(w["mean_velocity"]),
70             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
71             float(g.crossings),                       # net crossings (count) ?
72         ])
73     return intervals, X

```

34. user (2026-06-16T06:28:11.679Z)

```

=== expected.json for multi+occlusion (net_crossings + control_segments) ===
clean_single_rally [[0.033333, 3.966667]] [7]
multi_rally_with_deadtime [[0.033333, 4.0], [7.033333, 10.966667]] [7, 7]
occlusion_break [[0.033333, 3.966667], [7.033333, 10.966667]] [7, 7]
=== git log for windowing/golden PRs ===
8d3e185 feat(eval): golden fixtures P1 ? de-attribution/minimization tool + M2 refusal gate (#192)
64494ef feat(eval): golden regression fixtures M1 ? video-free CONTROL-only characterization harness (#189)
a3b209a feat(windowing): Tier 1 (W1) ? bounded windowing fixes the over-merge (?F1 +0.139, 6/0) (#187)
46ad601 fix(detectors): fail fast with an actionable error when a video source is unreadable from WSL (#177)
4251c9c fix(eval): close the eval?serve windowing gap ? single-source SERVED preset + served-regression promotion guard (#154)
7651687 feat: cache hygiene ? self-cleaning pipeline + cleanup tool + disk guard (#109)
31069cb fix(detectors): keep ~ expandable in WSL staging/inference commands (#97)
2c241a8 refactor: collapse the hybrid-segmenter twins into a shared trajectory base
c64a714 fix(reliability): bound WSL/provider timeouts, SQLite WAL+busy_timeout, chunk-step guard, CORS (#69)
34cdcfa fix(detectors): namespace WSL/cache/output artifacts by video_id, not basename (#64)
49272e5 feat(detector): finish DetectorRunner ABC (complete WSL de-risk seam)
7b9dad2 fix(tests): cross-platform support for Windows-style paths in eval batch resolver and tracknet/wasb basename logic
8615534 refactor: adopt approved layered directory structure

```

35. assistant (2026-06-16T06:28:17.520Z)

This is the central robustness finding. Let me now precisely characterize the divergence. The fixtures pin `max\_win=0.0` (W1 bounded-windowing OFF). The SERVED preset sources `max\_window\_duration` from `IndexingConfig` defaults (currently 0.0). But:

1. The replay path `build\_candidates(proposal=(vt,mg,mwd))` is a **3-tuple only** ? it physically **cannot** pass `max\_window\_duration` to `trajectory\_to\_action\_windows`. So the pinned `max\_win` in provenance is **decorative** in the replay ? it is read into manifest/provenance but the replay never consumes it (`\_pinned\_proposal` returns only 3 values, and `build\_candidates` has no `max\_win` param).

Let me verify exactly what `\_pinned\_proposal` does and that `max\_win` is never threaded into the replay, and confirm what would happen if W1 were turned on in SERVED defaults.

36. user (2026-06-16T06:28:19.234Z)

```

287     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
288         """The windowing proposal tuple, read from the fixture provenance when present (the

```

```

pinned
289         params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""
290         if prov and "windowing" in prov:
291             w = prov["windowing"]
292             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
293         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
294
295
296     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
297         """Replay one fixture through the shipped CONTROL path; return its recomputed
snapshot.
298
299         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
committed
300         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
301
302         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
shuttle
303         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
304
305         Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
306         control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
then
307         merges overlaps; NO numpy is touched.
308         """
309         fdir = _resolve_dir(slug_or_dir)
310         slug = fdir.name
311         traj_csv = fdir / "trajectory.csv"
312         labels_csv = fdir / "labels.csv"
313         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
else None
314
315         fps, fw = _fps_fw(prov, slug)
316         iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
317         proposal = _pinned_proposal(prov)
318
319         intervals, feature_rows = distill_local.build_candidates(
320             str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
321

```

37. user (2026-06-16T06:28:25.087Z)

```

=== IndexingConfig max_window_duration default ===
138:     max_window_duration: float = 0.0
=== windowing.build_windows callers (does golden use it?) ===
26: ``windowing.SERVED`` constant, so a future SERVED retune cannot silently churn fixtures.
69:#: here on purpose ? the spec forbids depending on the named ``windowing.SERVED`` constant so
a
289:     params travel with the fixture), else the module-level pinned defaults. NEVER
windowing.SERVED."""

```

38. assistant (2026-06-16T06:28:33.965Z)

Now I have confirmed the core robustness findings. Let me verify a couple more edge cases to be thorough and honest:

1. The `parse\_trajectory\_csv` swallows malformed rows silently (except (KeyError, ValueError): continue) ? but that's the detector path, and the fixtures are CONTROL synthetic so it's lower risk.
2. The `\_scan\_forbidden` substring scan for `"player"` ? does `\_round`/any field write "player"? And does `interval\_iou` mean_iou could be 0.983 (>1.0 active_ratio noted).
3. The W1 `\_split\_overlong\_groups` interaction: if SERVED's `max\_window\_duration` becomes > 0 in a future PR, the **replay would not change** (good for fixtures) but the production path WOULD ? meaning a fixture passing green tells you NOTHING about whether the W1 change is correct. This

is exactly the "passes silently" risk.

Let me confirm the `build_windows`` function (`windowing.py`) ? the doc §4a claims the replay uses `windowing.build_windows(preset)``, but the actual code uses `distill_local.build_candidates``. Let me verify this doc-vs-code divergence and check whether `min_split_gap`` is also unpinned.

39. user (2026-06-16T06:28:38.203Z)

```
=== Does replay touch windowing.build_windows? (doc 4a claims it does) ===
13:     parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
25:  provenance** and passed explicitly to ``build_candidates`` ? we do NOT rely on the named
302:     parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5 shuttle
319:     intervals, feature_rows = distill_local.build_candidates(

=== min_split_gap default + W1 split knobs ===
265:     max_window_duration: float = 0.0,
266:     min_split_gap: float = 0.5,
277:     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
282:     that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing fix
from
334:     if max_window_duration > 0:
336:         groups, int(max_window_duration * fps), int(min_split_gap * fps))

=== check: would 'player' substring ever be emitted by writers? scan FEATURE/key names ===
23:  person / pose / gait / face / audio field anywhere in the schema (positive-assertion test).
88:FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
323:     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
324:     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
508:     re-introducing a person/audio/pose stream or an id/path back-channel ? never the silent
default.
515:         f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player MUST
NOT "
577:     The committed bytes carry NO filename / video_id / match / player / capture-date ? only
an
```

40. assistant (2026-06-16T06:28:45.642Z)

I now have everything I need. Let me do a final check on two remaining edge cases: the `source_note`` leak scan when `time_origin_reset=False`` and frame `0`` handling, and whether the `_scan_forbidden`` source-path scan could be bypassed by a short token. Let me also verify whether the freeze_real path's `frame_width`` is recorded but a mismatch with the actual trajectory could go undetected, plus the schema\_version downgrade (only `>`` is rejected, equal/older accepted).

Let me confirm one determinism subtlety: `_fmt`/`_round`` use Python's `round()`` (banker's rounding), and floats are written at 6dp but compared at 1e-6 tolerance ? a value exactly at `0.5`` could round differently, but since both sides go through the same `_round`` it's consistent. The real cross-machine risk is in `build_candidates`` producing slightly different floats (e.g. `math.sin``, `0.5``). Let me check if any value sits right at the 1e-6 boundary.

41. user (2026-06-16T06:28:48.736Z)

```
=== schema_version guard: only > rejected (downgrade/equal accepted) ===
60:#: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
61:CURRENT_SCHEMA = 1
305:     Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}]},
343:         "schema_version": CURRENT_SCHEMA,
378:         "schema_version": CURRENT_SCHEMA,
641:         "schema_version": CURRENT_SCHEMA,
697:         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
732:         "schema_version": CURRENT_SCHEMA,
757:     whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible)."""
```

```

761:     sv = int(expected.get("schema_version", prov.get("schema_version", 0)))
762:     if sv > CURRENT_SCHEMA:
764:         f"{slug}: schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA} ? upgrade the
harness")
777:     TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be EXACTLY
794:     exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))

```

=== mean_iou 0.983 ? is that >1 active_ratio a concern? float fields near 1e-6 boundary ===
active_ratio=1.008475 (>1.0 ? active_frame_count/win_frames rounding; benign but note)

42. assistant (2026-06-16T06:29:05.660Z)

I have a complete and well-grounded picture. Let me synthesize the findings against the ROBUSTNESS lens.

Key confirmed facts:

1. **max_win** is pinned-but-ignored in the replay. **provenance.windowing.max_win** and **manifest.windowing.max_win** carry `0.0`, but `_pinned_proposal()` returns only `(vt, mg, mwd)` and `build_candidates()` has no `max_window_duration` parameter. The W1 bounded-windowing split (PR #187) is therefore **structurally unreachable** from the replay. If a future PR flips SERVED's `max_window_duration > 0` (or changes the W1 split algorithm), the production windowing changes but **the fixtures stay green** ? silent pass, not a loud fail. The pinned literal is genuinely safe against SERVED churn (good), but the cost is that the W1 dimension of the windowing algorithm is uncharacterized.
2. **The doc §4a overclaims the reuse map.** It states the replay path is `parse_trajectory_csv ? windowing.build_windows(preset) ? distill_local.build_candidates ? rally_gate net_crossings ? ...` and "Reuses verbatim ... `windowing.WindowingPreset` (default SERVED)". The actual code never imports or calls `windowing.build_windows`/`WindowingPreset`/`SERVED` ? it calls `distill_local.build_candidates` with a raw 3-tuple. So `windowing.py` is NOT on the replay path at all. A future refactor of `windowing.build_windows` or the `WindowingPreset` ? tuple bridge would not be caught by these fixtures, contradicting the doc.
3. **build_candidates** signature change handling. If `build_candidates`'s `proposal` 3-tuple shape changes (e.g. to accept `max_win` as a 4-tuple, or to take a `WindowingPreset`), the replay's `distill_local.build_candidates(..., proposal=proposal)` call breaks with a `TypeError`/`ValueError` ? that is a **loud** failure (good). If `trajectory_to_action_windows`'s `algorithm` changes (velocity formula, merge logic) the floats shift and the parse-compare fails loudly (good). If `rally_gate.estimate_play_axis`/`count_net_crossings` change, `net_crossings` (exact-int compared) fails loudly (good).

Let me confirm the remaining nuances around the net-crossings gate interaction with CONTROL and the FEATURE_NAMES assert.

The `FEATURE_NAMES` drift assert (line 82) is module-level ? if `distill_local.FEATURE_NAMES` changes, **import of golden_fixtures raises AssertionError**, which would break collection loudly. Good. The `net_crossings` feature IS computed in the replay and stored/compared, but CONTROL's `predict()` uses `min_crossings=0` so the gate is a no-op on segments ? meaning a change to the `rally_gate` **keep-decision** threshold wouldn't affect `control_segments`, but a change to the **crossing count** would still be caught via the exact-int `net_crossings` comparison.

I have enough to write the structured findings.

43. user (2026-06-16T06:30:16.919Z)

Structured output provided successfully

44. assistant (2026-06-16T06:30:29.472Z)

I've completed the robustness audit of the shipped Golden Regression Fixtures feature and returned my findings via StructuredOutput.

Key takeaways for the parent agent:

****The big miss (medium):**** ``max_window_duration`` / W1 bounded-windowing (PR #187 ? the windowing session the lens explicitly names) is pinned in ``provenance.windowing.max_win`` and ``manifest.json`` as ``0.0``, but is ****structurally unreachable from the replay****. ``_pinned_proposal`` (``backend/eval/golden_fixtures.py:287-293``) returns only ``(vt, mg, mwd)``, and ``distill_local.build_candidates`` (``distill_local.py:52-59``) has no ``max_window_duration`` parameter. So if a future PR flips SERVED's ``max_window_duration > 0`` (it sources from ``IndexingConfig``, currently ``0.0`` at ``config/models.py:138``) or changes the ``_split_overlong_groups`` algorithm (``tracknet_runner.py:334-369``), production windowing changes while the fixtures stay green ? the exact "passes silently (bad)" failure mode. This is a pinned knob the replay ignores, reading as coverage that does not exist.

****The good news (the lens's main worry is safe):**** the most-feared interactions fail LOUDLY. ``FEATURE_NAMES`` drift raises at import (``golden_fixtures.py:82-84``); a ``build_candidates`` proposal-shape change breaks the explicit 3-tuple call (line 319); windowing math changes diff via the 1e-6 parse-compare; and net-crossings is compared exact-int even though CONTROL gates at ``min_crossings=0``, so a crossing-count change (the #188/#190 gate) is caught. CONTROL touches numpy only via import, never execution.

****Doc overclaim (low):**** ``docs/GOLDEN_REGRESSION_FIXTURES.md:79-81`` says the replay goes through ``windowing.build_windows``/``WindowingPreset``/`"default SERVED"` ? none of which the code touches. The module docstring has the correct map; the doc is stale and self-contradictory with its own §4c.

Plus two forward-looking low-severity notes (schema_version has no downgrade/migration path once bumped; no row-count cap for P3 corpus growth + unvalidated ``frame_width`` in ``freeze_real_fixture``). Verdict: ship-safe for the committed synthetic tier; fix the ``max_win`` gap and §4a before W1 promotion or real fixtures land.